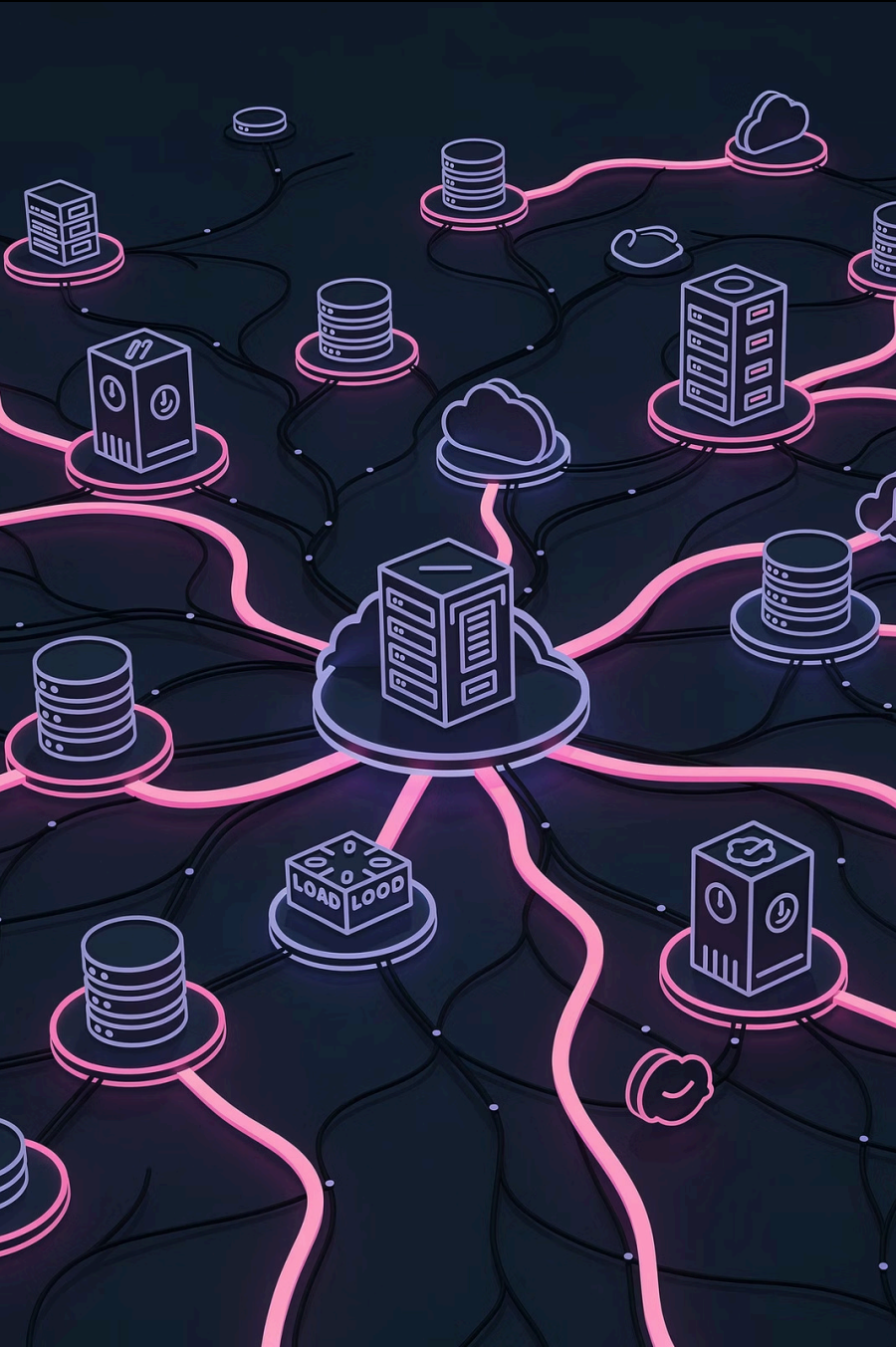




GenAI on AWS

AWS GenAI Service Landscape · Current State, February 2026

A technical deep-dive into Amazon Bedrock, SageMaker JumpStart, foundation models, and the new 2026 service tiers — everything you need to make informed GenAI architecture decisions on AWS.

Amazon Bedrock

A **fully managed service** giving access to **100+ foundation models** from leading AI providers — no infrastructure management required. Data is never shared with model providers, with encryption in transit and at rest.

Model Providers

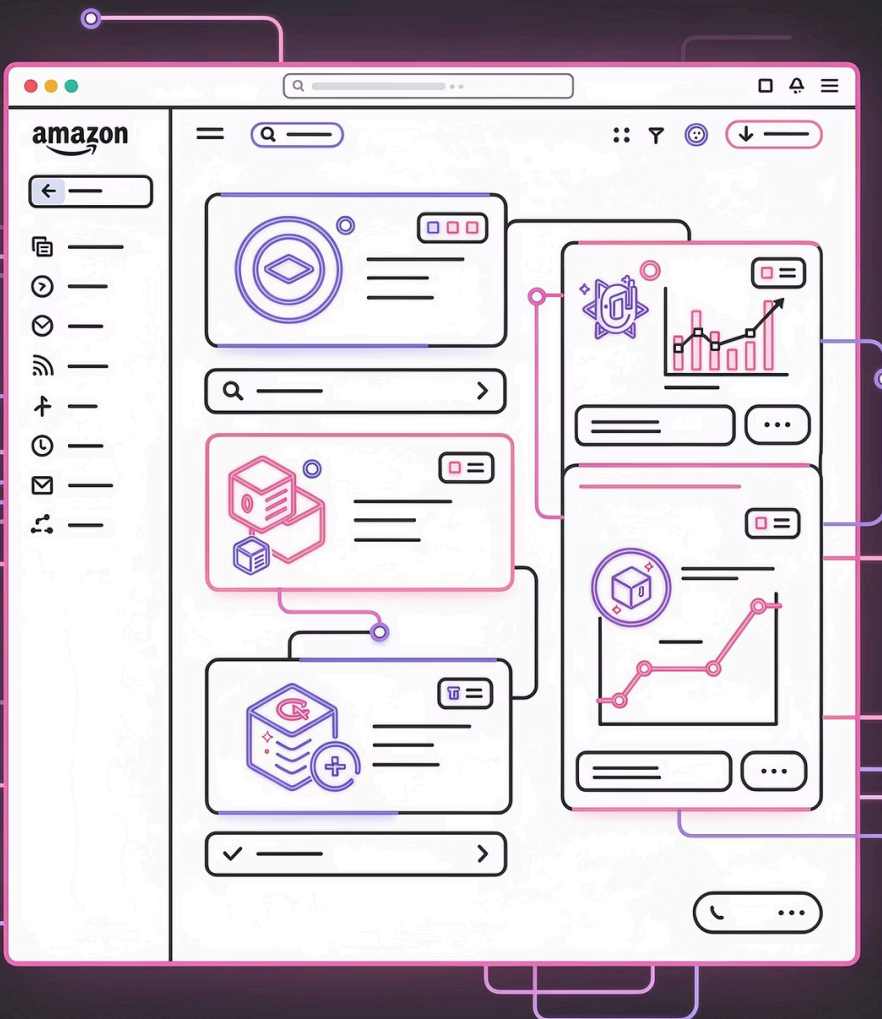
Amazon, Anthropic, OpenAI, Meta, Mistral AI, DeepSeek, AI21 Labs, Cohere, Stability AI, Qwen, MiniMax, Moonshot AI, Writer, and more. [docs.aws.amazon](https://docs.aws.amazon.com/bedrock/latest/userguide/providers.html)

2026 Additions

OpenAI gpt-oss-120b & gpt-oss-20b, DeepSeek V3.2, MiniMax M2.1, GLM 4.7, Kimi K2.5, Qwen3 Coder Next. [docs.aws.amazon](https://docs.aws.amazon.com/bedrock/latest/userguide/providers.html)

Enterprise Security

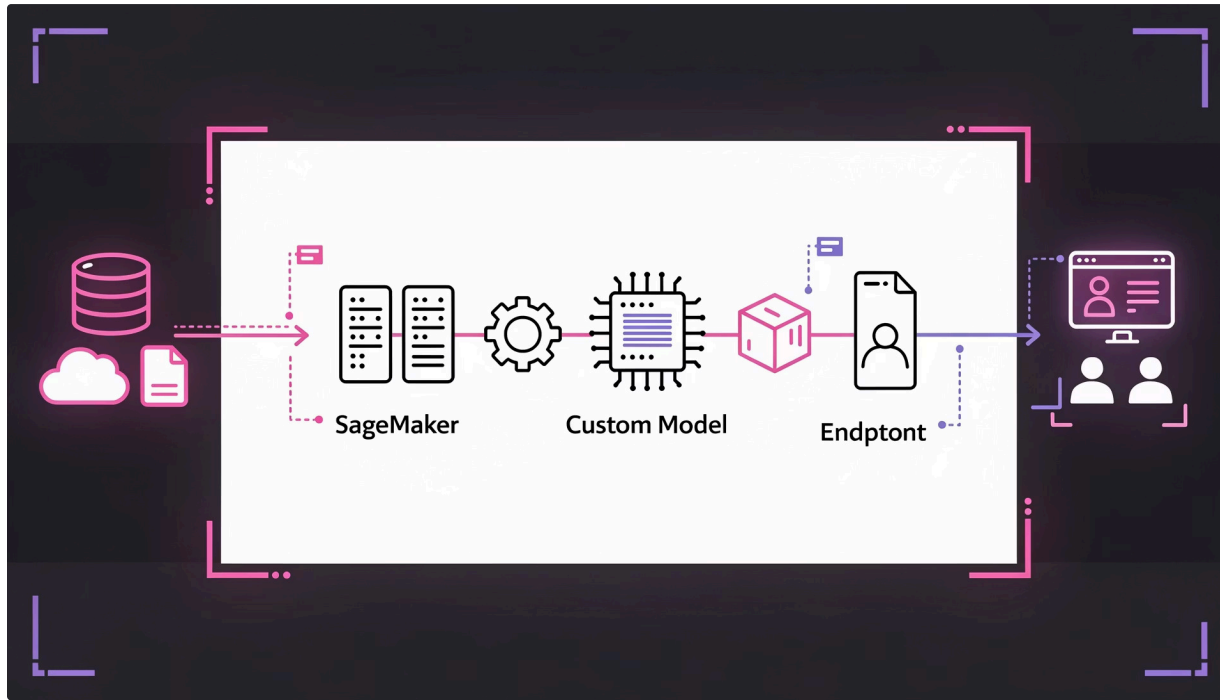
Built-in compliance with GDPR, HIPAA, SOC, and FedRAMP High — all via a unified API across every model. [docs.aws.amazon](https://docs.aws.amazon.com/bedrock/latest/userguide/providers.html)



ML DEEP DIVE

Amazon SageMaker JumpStart

Deploy and fine-tune open-source and proprietary models with **full infrastructure control** — ideal for teams with deep ML expertise who need to go beyond managed defaults.



Key Differentiators

- Full control over model deployment and compute
- Custom training and extensive fine-tuning
- Purpose-built ML pipelines beyond GenAI

Bedrock Integration

Models trained in JumpStart can be **registered directly with Bedrock**, unlocking Knowledge Bases, Guardrails, and the Converse API. [docs.aws.amazon](https://docs.aws.amazon.com/bedrock/latest/userguide/managed-models.html)

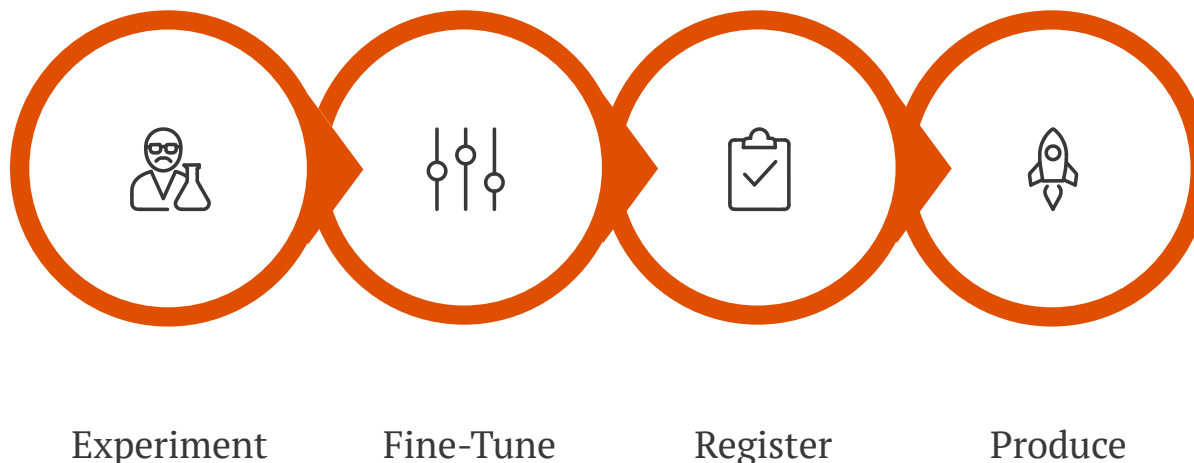
When to Use Each Service

Choose Amazon Bedrock

- Rapid production deployment without deep ML expertise
- Managed infrastructure with automatic scaling
- Enterprise-grade security and compliance out-of-the-box
- Multi-model experimentation and evaluation
- RAG systems, agents, and conversational applications

Choose SageMaker JumpStart

- Full control over deployment and infrastructure
- Extensive custom model training and fine-tuning
- Specialized ML requirements beyond managed services
- Cost optimization via custom infrastructure
- Complex ML pipelines that extend beyond GenAI



Use the services together: fine-tune custom models in JumpStart, then register them with Bedrock for production-grade managed features and consistent API access.

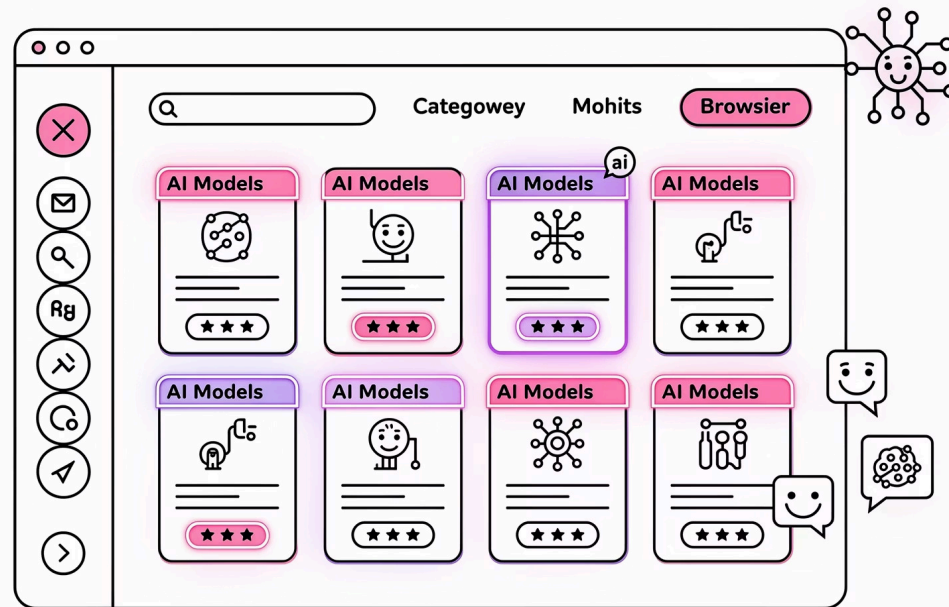
NEW IN 2026

Amazon Bedrock Marketplace

Discover, test, and subscribe to **100+ additional foundation models** beyond the core Bedrock catalog — all from a unified model discovery interface.

- Seamless integration with Bedrock Agents, Knowledge Bases, and Guardrails
- Managed endpoints with configurable scalability
- Expanded providers including Zendesk, IBM, and more

aws.amazon.com/bedrock/marketplace



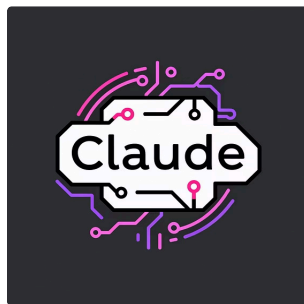
Foundation Models in Bedrock: Flagship Providers



Amazon Nova

Multimodal (text, image, video, documents) with Nova Pro & Premier for extended RAG workflows.

[aboutamazon](#)



Anthropic Claude 4.6

Latest Opus, Sonnet, and Haiku variants. Advanced reasoning, long context windows. Sonnet available in Reserved tier. [aboutamazon](#)



OpenAI (OSS)

gpt-oss-120b (117B params) and gpt-oss-20b — Mixture-of-Experts architecture, 128K context, Apache 2.0 license. [aws.amazon](#)

Foundation Models: Extended Provider Ecosystem

Meta Llama Family

Llama 3 through 4 series — 7B to 70B+ parameters, conversational and chat-optimized variants. [aws.amazon](https://aws.amazon.com/llama/)

Mistral AI

Mistral 7B general-purpose, Mistral 7B Instruct for chat, plus newer Ministral and Magistral models. [docs.aws.amazon](https://docs.aws.amazon.com/llama/)

DeepSeek V3.2

Advanced reasoning and agentic intelligence, optimized for complex multi-step workflows. [cloudprice](https://cloudprice.com/)

Coding Specialists

MiniMax M2.1 for autonomous coding, Qwen3 Coder Next for code generation, Kimi K2.5 for reasoning tasks. [docs.aws.amazon](https://docs.aws.amazon.com/llama/)

AI21, Cohere, Stability

Jamba 1.5 Mini/Large for language tasks; Command & Embed for retrieval; Stable Diffusion for image generation. [docs.aws.amazon](https://docs.aws.amazon.com/llama/)

GLM 4.7 & Flash

Cost-efficient models engineered for production deployment at scale. [docs.aws.amazon](https://docs.aws.amazon.com/llama/)

Bedrock Service Tiers (New in 2026)

Four tiers let you match every workload to the right balance of performance, availability, and cost. [aws.amazon](https://aws.amazon.com/bedrock/pricing/)

Reserved

Guaranteed capacity, 99.5% uptime SLA, 1–3 month commitment. Fixed token-per-minute pricing for mission-critical apps.



Priority

Up to **25% better OTPS latency** with preferential processing. No reservation required. Ideal for customer-facing chatbots.

Standard

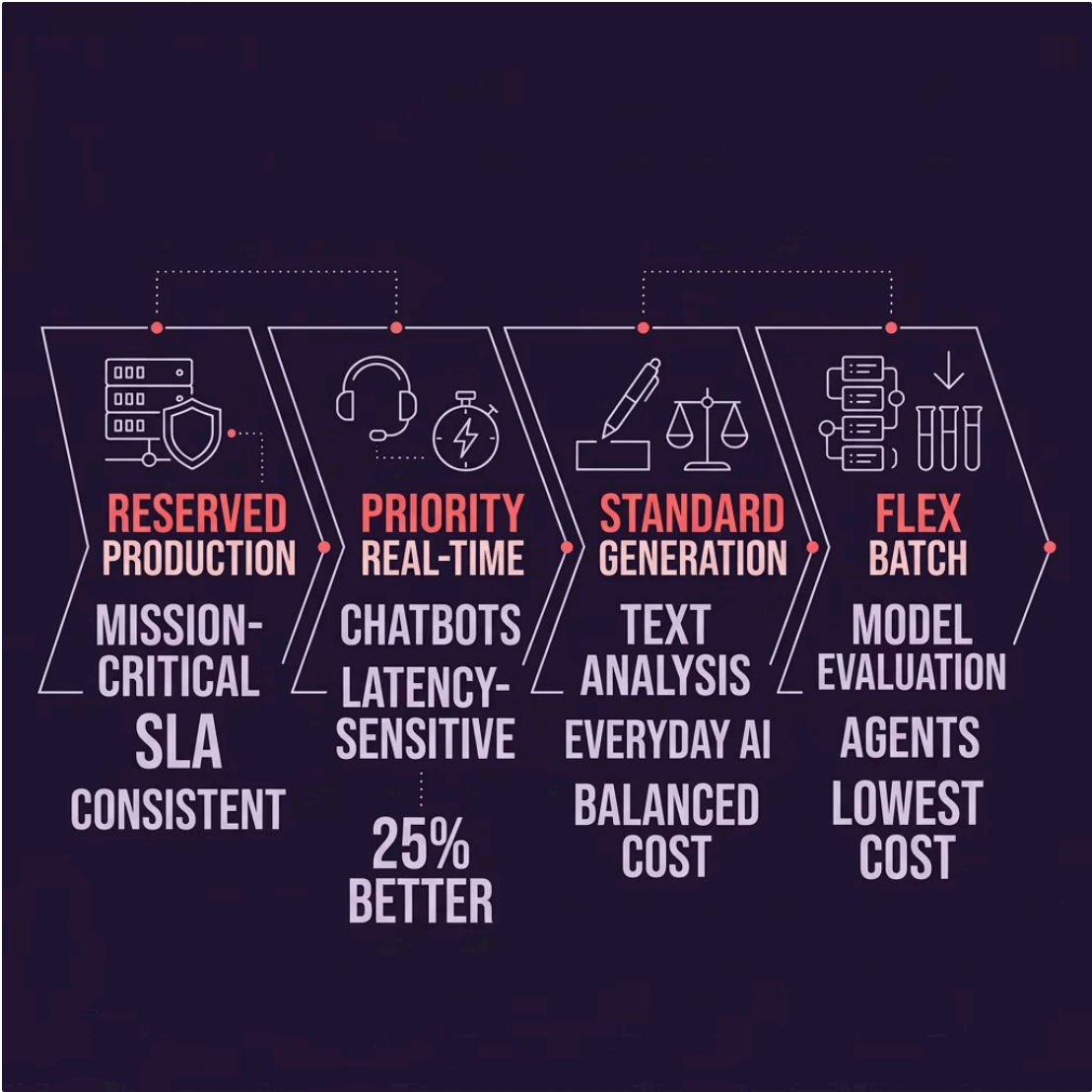
Default tier. Balanced cost and performance with moderate throughput guarantees. Best for everyday content generation tasks.



Flex

Lowest cost, best-effort availability, no SLA. Purpose-built for batch processing, model evaluations, and multi-step agentic workflows.

Matching Tiers to Workloads



Tier Selection Strategy

Choose your tier based on two primary axes: **latency sensitivity** and **cost tolerance**.

Cost Optimization

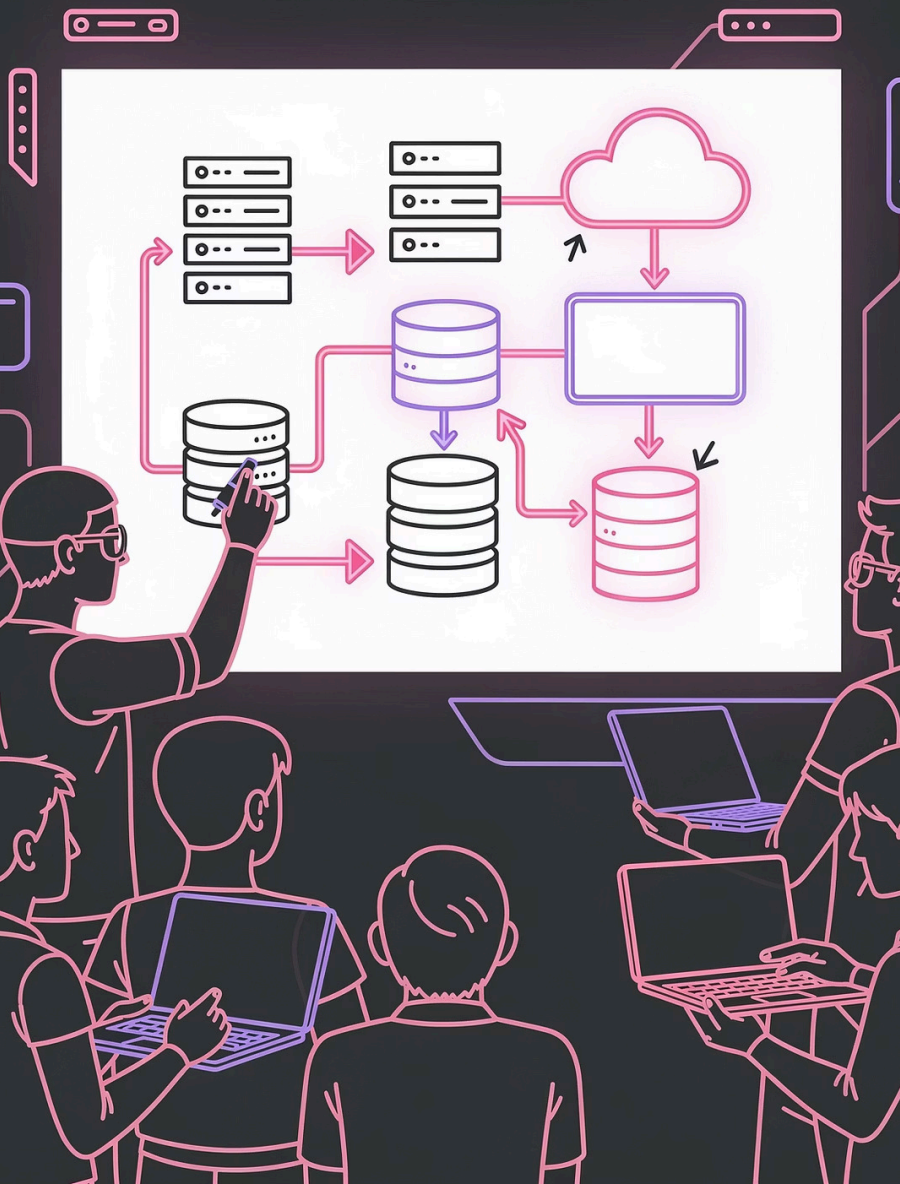
Route batch jobs and evaluations to Flex tier — savings can be substantial vs. Standard for non-urgent workloads.

Predictable Capacity

Reserved tier commitments eliminate throttling risk for production workloads with consistent volume requirements.

Hybrid Routing

Combine tiers per use case — Priority for real-time UX, Flex for background pipelines, in the same application.



Key Takeaways

1 Decision Framework

Bedrock for managed, production-ready apps; JumpStart when you need custom training, infrastructure control, or specialized ML pipelines.

2 Model Breadth

100+ models from 15+ providers — evaluate across dimensions like context length, modality, licensing, and latency profile before committing.

3 Tier Strategy = Cost Strategy

Map every workload to the right tier: Reserved for SLA guarantees, Priority for UX, Standard as default, Flex for batch and eval jobs. [aws.amazon](https://aws.amazon.com)

4 2026 Highlights

OpenAI OSS models, the Bedrock Marketplace, and new service tiers make AWS the most comprehensive managed GenAI platform available today.



Module 2: Getting to Know Amazon Bedrock

Research Findings — Current State, February 2026

MODULE 2

AMAZON BEDROCK

Amazon Bedrock Fundamentals

Amazon Bedrock is a **fully managed, serverless AI service** that gives teams immediate access to foundation models without the burden of infrastructure management. Its architecture is built for enterprise-grade reliability, security, and scale.

Serverless & Managed

No infrastructure to provision or maintain. AWS handles scaling, availability, and patching automatically.

Multi-Tenant Isolation

Strong data and compute isolation guarantees across tenants, with no cross-customer data exposure.

Regional by Design

Models and data remain within your selected AWS Region, supporting data residency and compliance requirements.

API-First

All capabilities are accessible via APIs and SDKs — fully automatable and integrable with existing pipelines.

Foundation Models

100+ models from Amazon, Anthropic, Meta, Mistral, OpenAI, and more — accessed through a **unified API surface**. Model weights are never exposed to your account for hosted FMs.

Model Access & Availability

Access is requested per model in the Bedrock console and granted at the account/Region level. Some models are immediately available; others require provider approval. Regional availability and service tier options vary by model.

API Patterns Deep Dive

Amazon Bedrock exposes **four distinct API patterns** in 2026, each optimized for different use cases — from OpenAI-compatible migration paths to native low-level model invocation.

1

Responses API

Recommended for new projects. OpenAI Responses API-compatible schema via `bedrock-mantle.{region}.api.aws`. Best for stateful conversations, agentic workflows, built-in tool use, and server-side tool execution. Supports multimodal inputs and streaming.

2

Chat Completions API

Best for OpenAI Chat Completions migrations. Available on both the OpenAI-compatible mantle endpoint and native runtime. Works with existing OpenAI-compatible SDKs. Ideal for stateless or application-managed multi-turn chat and text-focused tasks.

3

Converse API

Native Bedrock interface. Unified schema across Amazon, Anthropic, and other models via `bedrock-runtime.{region}.amazonaws.com`. First-class support for system prompts, tools, guardrails, and streaming via `ConverseStream`.

4

Invoke API

Maximum control. Direct model invocation with `InvokeModel` (synchronous) or `InvokeModelWithResponseStream` (streaming). Best for single-turn transactions, large custom payloads, and model-specific request schemas.

Python SDK (boto3) Basics

Installation & Authentication

Install the SDK with `pip install boto3 botocore`. Bedrock uses standard AWS credential mechanisms — IAM roles, named profiles, or environment variables. Ensure your IAM policy includes actions such as `bedrock:InvokeModel` and `bedrock:Converse`.

Two Clients, Two Planes

- **bedrock** client — control plane: model catalog, access management, fine-tuning jobs
- **bedrock-runtime** client — inference plane: Invoke, Converse, Chat Completions

Converse API Example

```
response = bedrock_runtime.converse(
    modelId="anthropic.claude-3-5-sonnet-v2",
    messages=[{
        "role": "user",
        "content": [{"text": "Hello!"}]
    }],
    inferenceConfig={
        "maxTokens": 512,
        "temperature": 0.5
    }
)
```

Client Setup

```
import boto3
region = "us-east-1"

# Control plane
bedrock_client = boto3.client(
    "bedrock",
    region_name=region
)

# Inference plane
bedrock_runtime = boto3.client(
    "bedrock-runtime",
    region_name=region
)
```

InvokeModel Example

```
import json
response = bedrock_runtime.invoke_model(
    modelId="anthropic.claude-3-5-sonnet-v2",
    body=json.dumps({
        "messages": [{
            "role": "user",
            "content": "Your prompt here"
        }],
        "max_tokens": 300,
        "temperature": 0.7
    })
)
```

Note: Request schema differs by model family (Anthropic vs. Nova vs. GPT-OSS). Always consult model-specific docs.

Model Selection Criteria

Choosing the right model in Bedrock requires evaluating **seven key dimensions**. There is no universal best model — the optimal choice is always workload-specific.



Modality

Choose text-only or multimodal models (text + image + document + video). Match input types — text, images, documents, audio — to expected outputs.



Model Class & Size

Light (7B–20B): fast and cheap for high-volume tasks. **General**: balanced quality vs. latency. **Flagship/Frontier**: best reasoning at higher cost. Use benchmarks, not just parameter counts.



Context Window

Ranges from 4K–8K (small models) to 32K–128K (modern LLMs) to 200K+ tokens (Claude variants). Larger windows increase cost but enable long-document RAG and complex histories.



Inference Latency

Real-time apps need faster models and higher service tiers. Batch/offline workloads can tolerate higher latency for better quality. Service tier (Priority / Standard / Flex) directly affects throttling risk.



Pricing

Billed separately for input and output tokens; output is typically more expensive. GPT-OSS models are significantly cheaper than flagship proprietary models. Always check the Bedrock pricing page for current Region-specific values.



Fine-Tuning & Custom Models

Selected Amazon Titan/Nova and partner models support fine-tuning or continued pretraining. Use **Custom Model Import** or SageMaker JumpStart to bring your own models while retaining Bedrock tooling.



Special Capabilities

Tool use/function calling, advanced reasoning (GPT-OSS, DeepSeek, Claude frontier), code generation (Qwen Coder, Nova), and multilingual support vary by model family.

Pricing Models (2026)

Bedrock offers **four complementary pricing levers** that, when used together, can dramatically reduce total inference cost while maintaining performance for the right workloads.

On-Demand

Pay per input/output token with no minimum commitment. Service tiers — **Priority**, **Standard**, and **Flex** — apply per model.

Example: Claude 3.5 Sonnet ≈ **\$3/1M input** · **\$15/1M output** tokens (Standard, one Region, subject to change).

Reserved Capacity

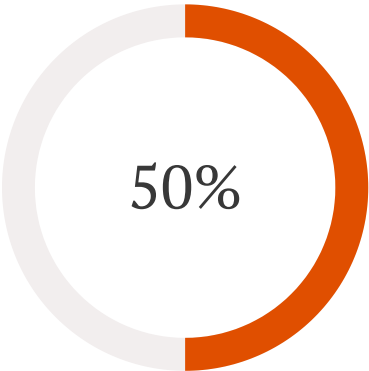
1- or 3-month commitments for specific models and tiers. Provides **guaranteed throughput** and tighter SLAs. Ideal when sustained usage is a large fraction of peak demand.

Batch Processing

Asynchronous jobs with S3 inputs/outputs processed over a multi-hour window. Delivers up to **~50% savings** vs. on-demand for eligible models. Best for content moderation, analytics, training data generation, and offline evaluations.

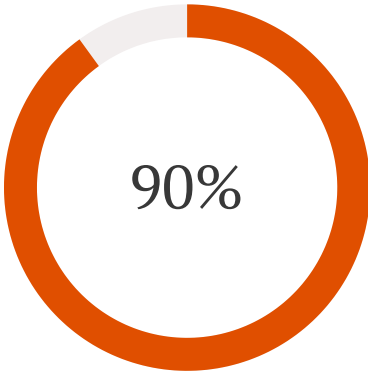
Prompt Caching

Reuse long prompt prefixes across many requests. Cache reads receive up to a **~90% discount** on input tokens. Currently supported on select Anthropic Claude and Amazon models, with coverage expanding.



Batch Savings

Approximate token cost reduction vs. on-demand for eligible batch inference workloads



Cache Discount

Potential discount on cached input tokens using prompt caching on supported models

Error Handling and Rate Limits

Building production-grade Bedrock applications requires anticipating and gracefully handling the four most common error types — and understanding how service tiers affect your quota headroom.

429 ThrottlingException Cause: Exceeded requests-per-second or tokens-per-second limits. Solution: Implement exponential backoff with jitter, reduce concurrency, or request a quota increase. Upgrade to Priority or Reserved tiers for higher limits and lower throttling risk.	503 ServiceUnavailableException Cause: Temporary capacity or service-side issues. Solution: Retry with backoff. For critical workloads, architect for cross-Region inference failover to maintain availability during regional disruptions.
400 ValidationException Cause: Invalid parameters or malformed payload structure for the target model or API. Solution: Validate request schemas carefully. Consult model-specific docs for max token limits, required fields, and payload structure differences across model families.	403 AccessDeniedException Cause: Missing IAM permissions or model access not granted for the account/Region. Solution: Update IAM policies to include required Bedrock actions and confirm model access is enabled in the Bedrock console for the correct account and Region.

Rate Limit Tiers at a Glance

1	2	3
Flex Lowest RPM/TPM quotas. Best for experimental and non-urgent workloads.	Standard Moderate quotas and balanced throttling. Default choice for most production workloads.	Priority / Reserved Highest quotas, lowest throttling risk. Designed for latency-sensitive, mission-critical traffic.

Monitoring and Observability

CloudWatch Metrics (Automatic)

Bedrock automatically emits the following metrics for every model invocation — no additional configuration required to start collecting them:

- **Invocations**

Total number of requests sent to a model endpoint.

- **InvocationLatency**

End-to-end latency per request — critical for SLA monitoring.

- **InvocationThrottles**

Count of throttled requests; a key signal for tier upgrade decisions.

- **InputTokenCount / OutputTokenCount**

Token volumes for cost attribution and budgeting.

- **4xx / 5xx Error Counts**

Client errors (validation, auth) vs. server errors (capacity, service issues).

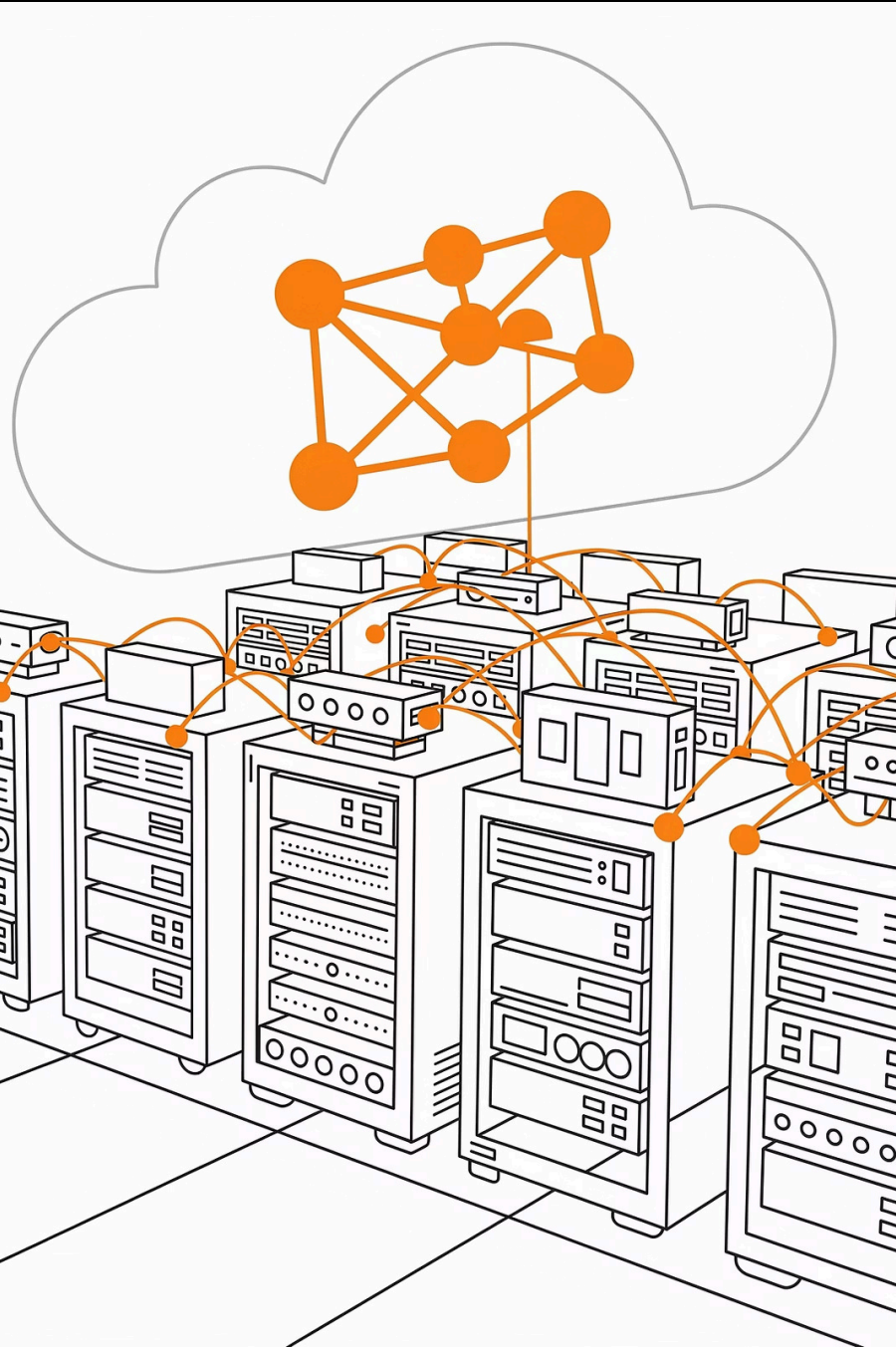
Model Invocation Logging

Beyond metrics, enable **Model Invocation Logging** to capture rich per-request data. Logs can be routed to **CloudWatch Logs**, **Amazon S3**, or both.

Log records can include:

- Full request and response payloads
- Input and output token counts
- Timestamps and request metadata
- Model ID and invocation parameters

Use invocation logs for **debugging**, **safety reviews**, **quality evaluation**, and **cost analysis**. Combine with CloudWatch dashboards for end-to-end visibility from day one.



Module 3

RAG on Bedrock

Current State — February 2026 | Amazon Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases Overview

Amazon Bedrock Knowledge Bases is a fully managed RAG (Retrieval Augmented Generation) capability that orchestrates data ingestion, retrieval, and generation across supported data sources and vector stores — all without requiring teams to build and maintain a custom pipeline.

Managed Connectors

Native integrations for listed data sources, plus programmatic ingestion APIs for custom sources.

Auto Embeddings

Automatically converts ingested documents into vector embeddings using your selected embedding model.

Multi-Source Support

S3, Confluence, SharePoint, Salesforce, Web Crawler (preview), and more via configuration.

Vector DB Flexibility

Choose from AWS-native or third-party managed vector stores supported by Knowledge Bases.

Built-in Security

IAM integration, encryption at rest and in transit, VPC support, and AWS-native security controls.

RAG Architecture: Data Sources (2026)

Unstructured — Native Connectors

→ Amazon S3

PDFs, HTML, Office docs. Incremental sync, metadata fields, prefix/suffix filters.

→ Confluence

Enterprise wiki content. Scheduled or on-demand sync via native connector.

→ SharePoint

Document libraries and sites with permission-aware access.

→ Salesforce

CRM records and knowledge articles, respecting object and field-level permissions.

→ Web Crawler (Preview)

Crawl public web content with configurable depth, domain restrictions, and URL filtering.

Structured / Graph / Enterprise Search

Amazon Kendra / GenAI Index

Leverage existing enterprise search indexes as a Knowledge Bases retrieval backend. Enables hybrid keyword + semantic search over enterprise content.

Amazon Neptune Analytics

Build **GraphRAG** solutions on graph-structured data — multi-hop traversal, entity linking, and relationship-aware retrieval.

Natural Language → SQL

Supported as an **agent pattern** via Agents + Lambda + SQL endpoints (RDS/Aurora). Not a direct KB data source type.

📄 For sources not natively supported, use programmatic ingestion APIs to push pre-processed documents and embeddings directly into a Knowledge Base.

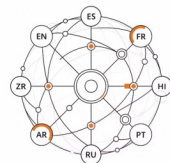
Embedding Models

The choice of embedding model directly impacts retrieval quality, latency, and cost. Bedrock Knowledge Bases supports Amazon-native, Cohere, and custom/third-party models to address diverse use cases.



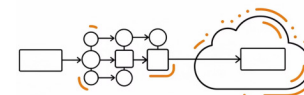
Amazon Titan Embeddings

- Titan Text Embeddings for text-only content
- Titan Multimodal Embeddings for text + image in multimodal Knowledge Bases
- ~\$0.0001 per 1K tokens (region-dependent; see pricing page)
- Best for general-purpose, cost-effective semantic search



Cohere Embed Models

- English-focused and multilingual models via Bedrock
- 100+ language support; dimensions range 768–4096
- Well-suited for cross-lingual and international applications



Custom & Third-Party Embeddings

- Run custom models on SageMaker endpoints
- Generate embeddings externally and ingest via programmatic APIs
- Hugging Face and other sources integrated via SageMaker — not available as direct drop-down options in the KB UI

Vector Databases: Supported Options

Knowledge Bases supports multiple AWS-native and third-party vector stores as retrieval backends. The right choice depends on your existing infrastructure, workload profile, and operational preferences.

Amazon OpenSearch Serverless

Fully managed, no cluster sizing. Auto-scaling OCU's for variable workloads. k-NN ANN retrieval + hybrid keyword/semantic search.

Best for: AWS-native deployments with fluctuating query volumes.

Amazon OpenSearch Service (Managed Clusters)

Fine-grained control over cluster config and performance tuning. Access to advanced ML features. **Best for:** Enterprises already running OpenSearch who need tuning control.

Amazon Aurora PostgreSQL (pgvector)

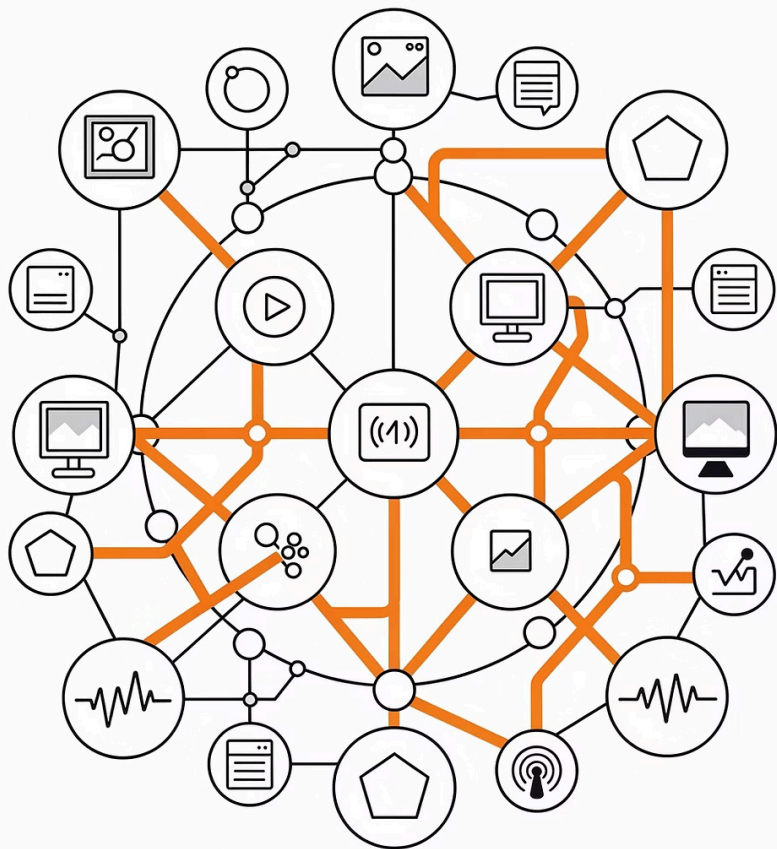
Relational + vector via pgvector extension. ACID transactions and SQL queries alongside embeddings. **Best for:** Teams already on PostgreSQL who want to add vector search.

Amazon Neptune Analytics

Graph-native store for GraphRAG. Multi-hop graph traversal and relationship-aware retrieval. **Best for:** Knowledge graphs and complex entity relationships.

Third-Party Stores

Pinecone — purpose-built, high-performance vector search at scale. **Redis Enterprise Cloud** — low-latency in-memory for real-time use cases. **MongoDB Atlas** — document + vector with flexible schema and global distribution.



Advanced RAG Features (2026)



GraphRAG (Neptune)

Multi-hop reasoning across entities and relationships. Better entity linking, disambiguation, and explainable retrieval paths. Use cases: fraud networks, biomedical knowledge, complex document interpretation.



Multimodal RAG

Retrieve and ground answers on text, images, audio, video, and document layouts using Titan Multimodal Embeddings + Amazon Nova. Handles annotated docs, diagrams, charts, and slide decks.



Reranking Models

A reranker model scores candidate chunks against the query, reordering the top-k results for higher precision. Use Cohere rerankers or LLM-based rerank patterns. Particularly valuable for ambiguous queries.



Hybrid Search

Combines keyword/lexical and semantic vector search. Better recall on specific terms, IDs, and structured text. Natively supported in OpenSearch (serverless and managed) and via Kendra integration.

Chunking Strategies

How you split documents into chunks is one of the most impactful decisions in a RAG pipeline — it determines context coherence, retrieval precision, and ultimately generation quality.

Default / Automatic Chunking

Knowledge Bases automatically parses and chunks documents using a fixed-size with overlap strategy — roughly in the **few-hundred token range** with configurable overlap.

Effective for most generic use cases out of the box with minimal configuration required.

Advanced Chunking Options

1 Semantic Chunking

Split along semantic units — paragraphs, sections, headings — to preserve coherent context within each chunk.

2 Hierarchical Chunking

Represent parent-child document structure (document → section → paragraph) via metadata or graph structures for context-aware retrieval.

3 Custom Chunking

Pre-process documents via Lambda or offline scripts, then push via programmatic ingestion APIs. Full control over split logic.

4 Metadata Preservation

Maintain document title, section, page number, language, and version with each chunk for richer filtering and interpretability.

RAG APIs

Knowledge Bases exposes two core APIs that support different levels of control — from pure retrieval to fully managed end-to-end RAG in a single call.

Retrieve API

Purpose: Retrieve relevant chunks or documents *without* running generation — giving you full control over the generation step.

When to use:

- Custom or model-specific generation logic outside Knowledge Bases
- Hybrid systems where retrieval and generation are decoupled
- Debugging retrieval quality and coverage independently

Returns: Ranked list of retrieved contexts with relevance scores, source metadata, and multimodal references (when enabled).

RetrieveAndGenerate API

Purpose: End-to-end RAG in a single API call — retrieval plus generation in one managed workflow.

When to use:

- Rapid implementation of Q&A bots, assistants, and knowledge agents
- Standardized RAG flows requiring minimal orchestration code

Features: Custom prompt templates, selectable foundation model, configurable RAG parameters, and grounded responses with citations (text and multimodal).

📌 Use the **Retrieve API** when you need flexibility and debugging control. Use **RetrieveAndGenerate** when you want a production-ready, low-code RAG endpoint.

RAG Evaluation (2026)

Knowledge Bases includes built-in **RAG evaluation** features covering both retrieval-only and end-to-end retrieval + generation scenarios, powered by LLM-as-a-Judge scoring.

Retrieval Evaluation Metrics

Context Relevance

Are retrieved chunks relevant to the user's query?

Coverage

Do retrieved chunks collectively cover the important aspects of the query?

Used to tune chunking strategy, embedding model selection, top-k, filters, and hybrid search parameters.

Generation Evaluation Metrics

Quality

Correctness vs. ground truth, **Completeness** of the answer, and **Faithfulness/Groundedness** — is the answer supported by retrieved context?

Responsible AI

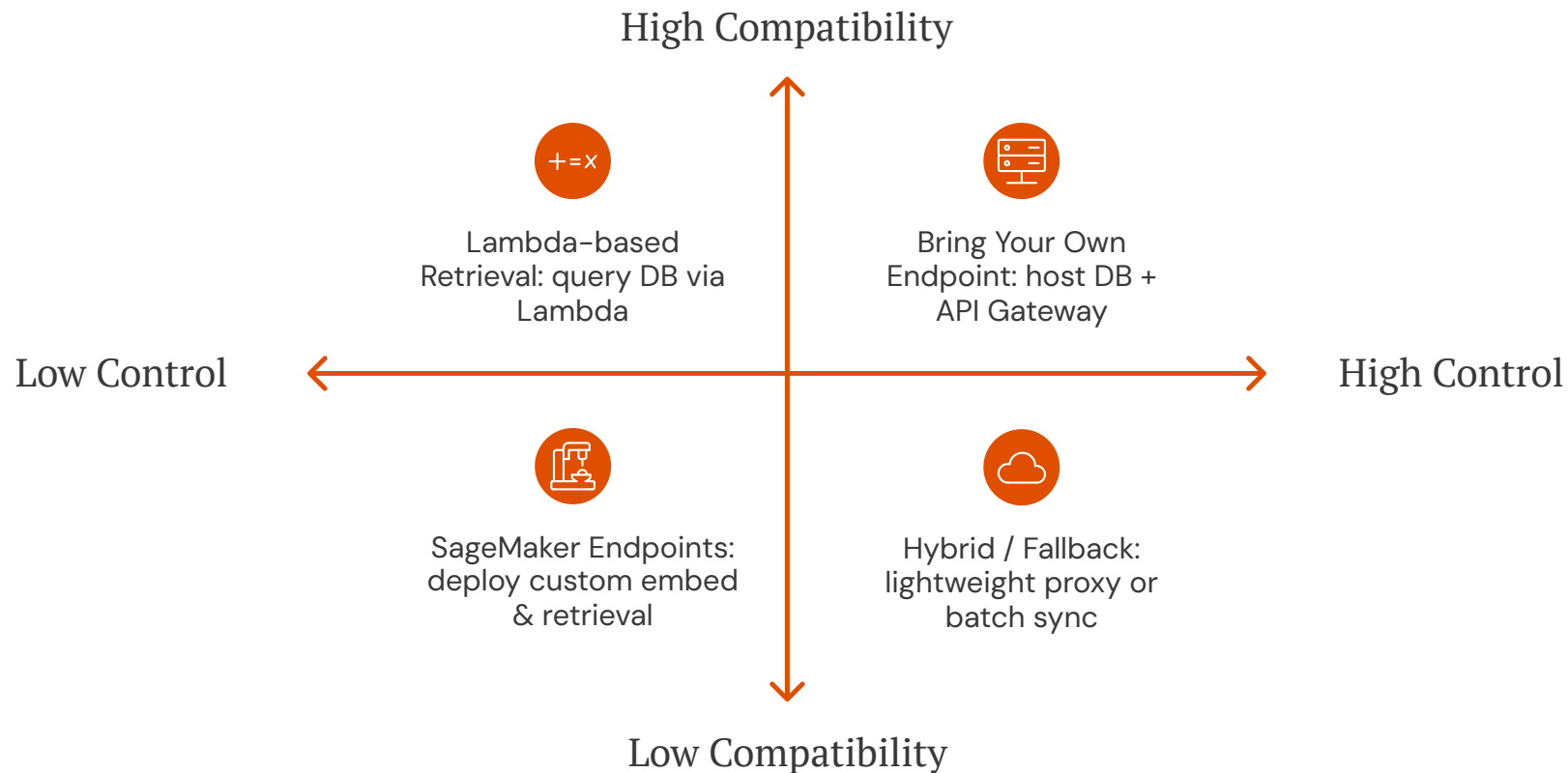
Harmfulness detection, **Appropriate refusal** behavior, and **Stereotyping/bias** checks.

Evaluator models include Anthropic Claude, Amazon, and Meta models. Metrics and prompts are configurable for domain and risk tolerance.



Integrating Vector DBs Not Natively Supported

Knowledge Bases maintains a **fixed list of supported vector stores**. For external databases not on that list, teams use custom architectures that pair Bedrock models with any vector DB of their choice.



These patterns do not register the external store inside Knowledge Bases directly, but they fully enable RAG architectures using Bedrock inference APIs (Converse, InvokeModel, Responses) with any vector database — at the cost of additional custom code and operational overhead compared to the managed KB store list.

Improving RAG Quality



Embedding Model Selection

Domain-tuned models can improve recall/precision by **20–40%** per community benchmarks. Consider domain specificity, multilingual needs, and dimension size vs. cost trade-offs. Best practice: run small evaluations on your own corpus using built-in RAG evaluation tools.



Chunking Strategy

300–500 token chunks with 10–20% overlap is a strong starting heuristic. Split along semantic boundaries. Propagate key metadata (title, section, page, language, version) with every chunk.



Retrieval Optimization

Tune top-k, similarity thresholds, and hybrid weights. Apply **metadata filters** (doc type, product, region) to narrow search space. Introduce reranking on the candidate set to improve final ordering and precision.



Data Quality

Remove boilerplate, duplicates, and stale content. Maintain structured metadata and consistent tagging. Schedule regular syncs and use versioning for corpus A/B testing and rollback capability.



Prompt Engineering

Use system prompts to constrain tone and grounding ("use only provided context"). Add few-shot examples demonstrating ideal citation behavior. Keep prompts concise but include explicit citation requirements.

Cost Optimization for RAG

RAG workloads on Bedrock incur costs across three primary dimensions: **storage**, **embedding/ingestion**, and **per-query retrieval + generation**. Understanding each layer is essential to managing total cost of ownership.

Cost Dimensions

Vector Storage

OpenSearch Serverless: OCUs + storage.
Aurora: instances + storage + I/O. Third-party (Pinecone, Redis, MongoDB): vendor pricing per index, vector count, and capacity.

Per-Query Costs

Vector-store query + optional reranking model inference + generation FM cost. Tune top-k, use filters, and restrict multimodal retrieval where not needed.

Embedding / Ingestion

One-time cost per document/chunk at ingestion time. Use incremental sync to pay only for new/updated docs. For large backfills, use batch inference pricing.

Optimization Strategies

→ Cache Frequent Queries

Implement response caching at the app or API layer. Use Bedrock prompt caching for repeated prompt prefixes on supported models.

→ Right-Size Chunks & Top-k

Optimize chunk size and retrieved chunk count to hit acceptable quality at minimum token and compute cost.

→ Choose Cost-Efficient Embeddings

Use smaller or cheaper embedding models where quality remains sufficient for your use case.

→ Batch Document Updates

Group re-embedding jobs to leverage batch inference pricing and reduce per-call overhead.

→ Archive Cold Data

Maintain only necessary embeddings in the active vector store — archive or remove low-value or stale content.

Key Takeaways for Workshop Content

01

Managed RAG Benefits

Show how Knowledge Bases handles connectors, ingestion, embeddings, vector storage, retrieval, and RetrieveAndGenerate — so teams can focus entirely on business logic rather than pipeline plumbing.

02

Vector Store Selection

Build a decision matrix comparing OpenSearch (Serverless vs. Managed), Aurora pgvector, Neptune Analytics, and third-party stores across latency, scale, cost, and existing infrastructure investments.

03

Embedding Model Impact

Demonstrate retrieval quality differences across embedding models on the same corpus using built-in RAG evaluation tools. Make the performance vs. cost trade-off tangible with real numbers.

04

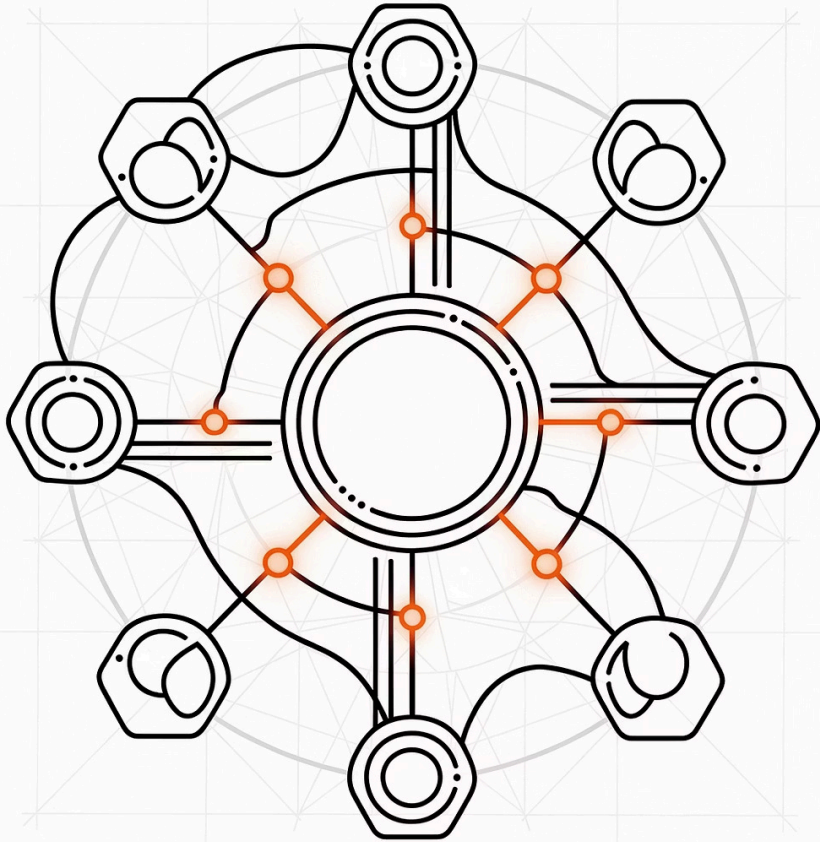
Evaluation Framework

Use Knowledge Bases evaluation (LLM-as-a-Judge) to teach participants how to measure and iteratively improve both retrieval precision and generation quality with configurable metrics.

05

Cost Awareness

Break down storage, embedding, retrieval, and generation costs line by line. Show how configuration choices — chunk size, top-k, model selection — directly impact the monthly bill.



Module 4: Agents on Amazon Bedrock

Research Findings — Current State, February 2026

AMAZON BEDROCK AGENTS

MODULE 4

Amazon Bedrock Agents Overview

Amazon Bedrock Agents enable developers to build intelligent, autonomous agents that go far beyond simple question-answering. These agents understand natural language instructions, decompose complex goals into actionable steps, and call external systems to get work done.



Natural Language

Understand and interpret complex user instructions without rigid command structures.



Task Decomposition

Break multi-step goals into discrete, executable sub-tasks with reasoning.



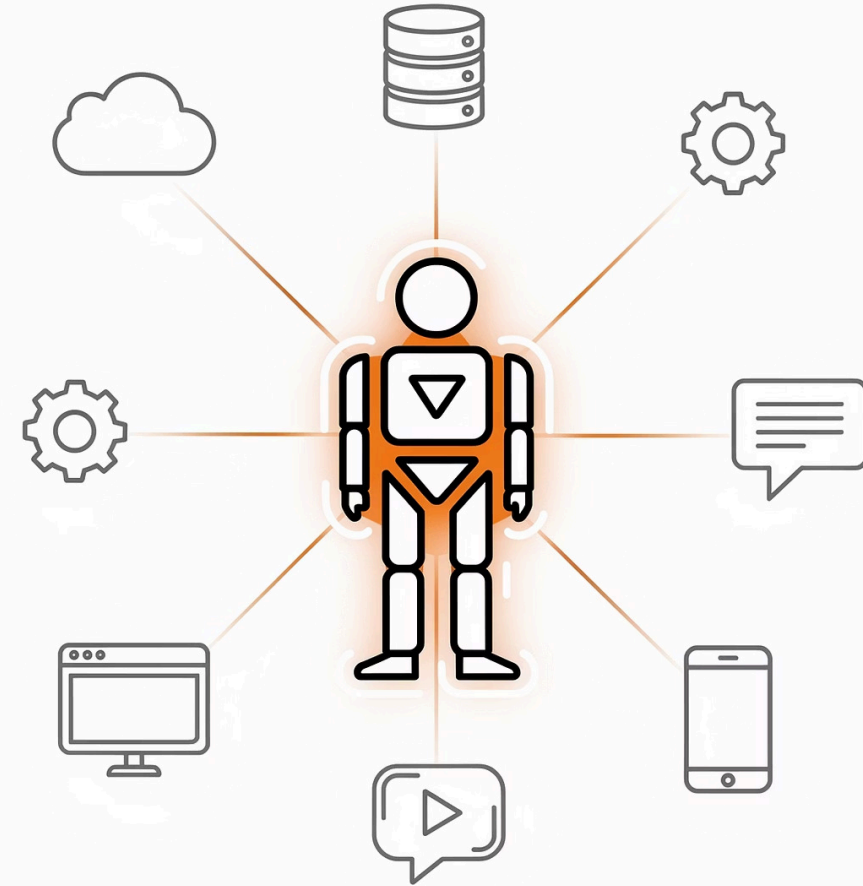
Tool & API Calls

Invoke external APIs, Lambda functions, and MCP-connected services on demand.



Grounded Answers

Use Knowledge Bases for retrieval-augmented generation and factually grounded responses.



Agent Architecture Components

Every Amazon Bedrock Agent is assembled from four core components. Understanding each layer is essential for designing reliable, production-ready agentic systems.

1

Agent Instructions

Define the agent's role, scope, and behavioral constraints. Be specific about purpose, define boundaries clearly, include edge-case examples, and specify tone, style, and safety guardrails.

2

Action Groups

Collections of callable actions backed by OpenAPI 3.0 schemas or function definitions, executed via Lambda or HTTP endpoints. Natural-language descriptions guide model selection. Refer to the Bedrock quotas page for current per-region limits.

3

Knowledge Bases

Attach one or more Bedrock Knowledge Bases for RAG. Answer questions from enterprise documents, provide grounded context, and combine retrieval with live actions like ticket creation.

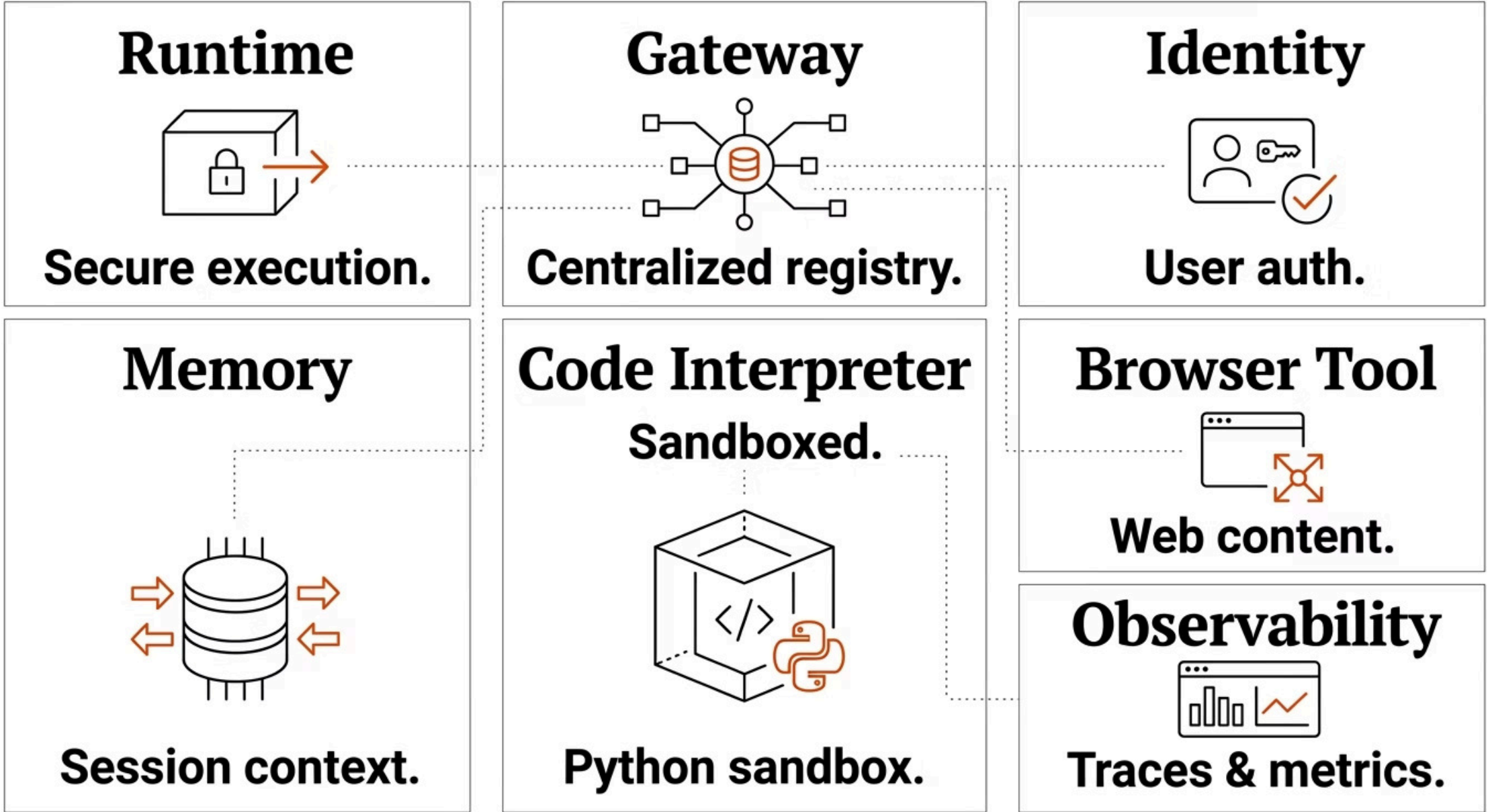
4

Foundation Model

Choose the FM driving agent reasoning and planning. Supported models include Amazon Nova, Anthropic Claude, Meta Llama, and GPT-OSS. Key trade-offs: reasoning quality, latency, multimodal capability, and context window size.

Amazon Bedrock AgentCore (2026)

AgentCore is AWS's production-grade suite of capabilities for building, operating, and scaling AI agents. It addresses the hardest infrastructure challenges — runtime reliability, tool integration, identity, memory, and observability — so engineering teams can focus on agent logic.



AgentCore Runtime

Enterprise-grade, secure, scalable runtime for production agents. Features automatic load-based scaling, session isolation across users and conversations, and deep integration with Bedrock service tiers and quotas.

AgentCore Gateway

Centralized tool and API gateway with first-class MCP support. Unites multiple MCP servers behind a single endpoint, translating them into Bedrock-compatible tools. Handles auth, throttling, versioning, and cross-agent tool reuse — reducing $M \times N$ integration complexity to $M+N$.

AgentCore Identity

Propagates user identity from the calling application all the way into agent tool calls. Enables fine-grained, per-user or per-tenant access control, session-level permission scoping, and full audit trails for compliance.

AgentCore Memory

Gives agents persistent context beyond a single session. Stores conversation history, user preferences, and past task outcomes. Enables personalization and continuity across channels — essential for customer support, sales, and productivity copilots.

AgentCore Code Interpreter

Sandboxed Python execution for data analysis, CSV/JSON transformation, and chart generation. Results are returned to the client or saved to S3 — enabling numeric reasoning inside live agent workflows.

AgentCore Browser Tool & Observability

Browser Tool

Enables agents to navigate web pages, retrieve HTML/text content, and extract information from documentation or knowledge sites that are not indexed in a Knowledge Base. Supports research, data gathering, and live doc reading. Not a full RPA system — limited automation patterns apply.

Observability

Provides step-by-step execution traces for every agent invocation — tool calls with inputs, outputs, and timing; decision-point reasoning; latency breakdowns; and token usage. Integrates with CloudWatch for production monitoring, alerting, and debugging.



Agent Development Approaches

Bedrock Agents support three complementary action-definition strategies. Choose the one that best matches your team's existing contracts and tooling.

1

OpenAPI-Based Actions

Define actions via OpenAPI 3.0 schemas describing REST-style APIs. Best for teams with existing REST APIs, complex request/response structures, or established OpenAPI contracts.

2

Function-Based Actions

Define actions around functions with structured parameters and return types. Ideal for simple utilities, direct LLM-to-function mapping, or rapid prototyping without OpenAPI overhead. Implement logic in Lambda or your own service.

3

Model Context Protocol (MCP)

Standardized protocol that acts as a universal translator for tools and data sources. MCP servers register with AgentCore Gateway, which exposes them as action groups. Scales M×N integrations to M+N — composable, secure, and reusable across agents and organizations.

📌 When using MCP, AgentCore Gateway translates MCP server capabilities into Bedrock-compatible tools automatically — no per-agent wiring required.

Lambda Integration: Best Practices

AWS Lambda is the primary backend for Bedrock Agent action groups. A clean Lambda integration requires the right IAM setup, predictable input/output formatting, and disciplined function design.

1 Keep Functions Focused

One clear responsibility per action. Avoid "god functions" that do too much — they're harder to debug and reason about.

2 Validate All Inputs

Treat agent-provided parameters as untrusted. Validate types, ranges, and required fields before executing any logic.

3 Handle Errors Gracefully

Return structured, meaningful error messages the agent can interpret and relay — not raw stack traces.

4 Set Timeouts & Log Everything

Use conservative Lambda timeouts. Log inputs, outputs, and context for debugging and auditing — scrubbing sensitive data as required.

Conceptual Lambda Handler

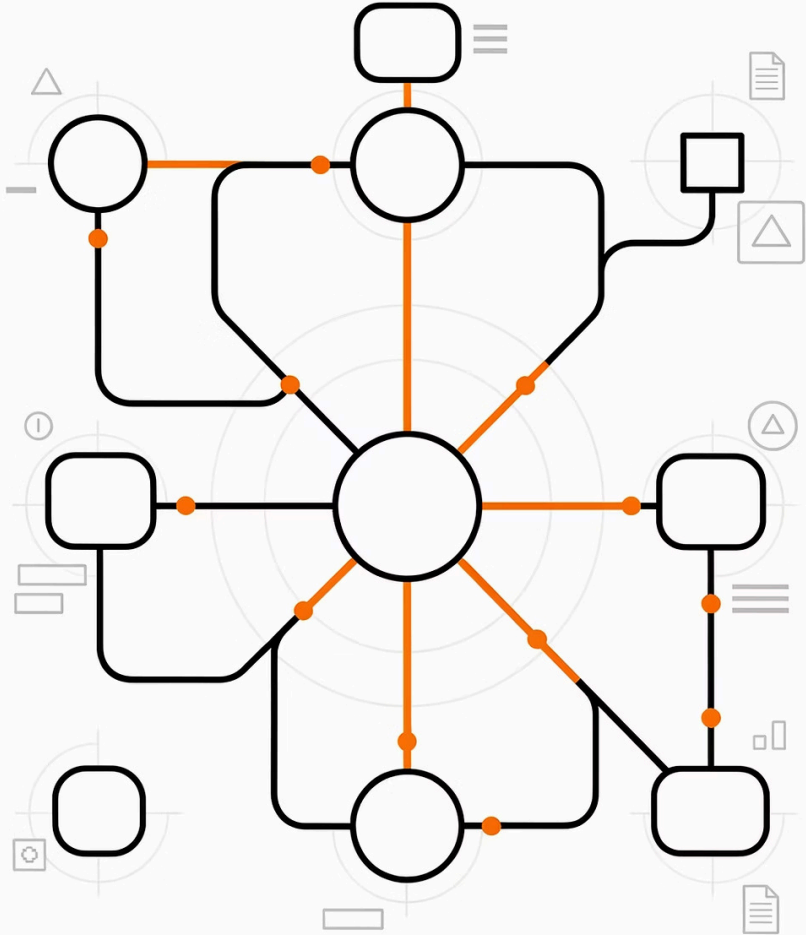
```
import json

def lambda_handler(event, context):
    action_group = event.get("actionGroup")
    function = event.get("function")
    parameters = event.get("parameters", {})

    # Dispatch to internal logic
    result = execute_action(function, parameters)

    return {
        "response": {
            "actionGroup": action_group,
            "function": function,
            "functionResponse": {
                "responseBody": {
                    "TEXT": {
                        "body": json.dumps(result)
                    }
                }
            }
        }
    }
```

This pattern matches the Bedrock Agents contract for returning tool results from a Lambda action group backend.



Multi-Agent Architectures

As workloads grow in complexity, a single agent is rarely enough. Multi-agent architectures distribute responsibility, improve maintainability, and allow specialized agents to excel at narrow tasks.



Sequential

Agents work in a pipeline — output of one feeds the next. Example: Research → Analysis → Report generation.



Parallel

Multiple agents work simultaneously on different sub-tasks. Results are aggregated at the end — ideal for multi-source data gathering.



Hierarchical

A supervisor agent delegates tasks to sub-agents based on type and complexity. Classic pattern: customer service router dispatching to billing, tech, or sales agents.



Collaborative

Agents share a common context and contribute partial solutions. Best for complex problems requiring negotiation or complementary expertise.

Agent Design Best Practices (2026)

01

Start Small, Define Clearly

Begin with a single, well-scoped use case. Expand capabilities incrementally once the core loop is stable and validated.

03

Build a Deliberate Tooling Strategy

Design tools for agent consumption: clear names, unambiguous descriptions, and well-typed schemas. Validate inputs and handle errors predictably.

05

Decompose Complexity

Break complex workflows into smaller steps or separate agents with clear interfaces between them.

02

Instrument from Day One

Enable AgentCore Observability and CloudWatch metrics from the first deployment. Log tool invocations, token usage, latencies, and errors before you need to debug them.

04

Automate Evaluation

Create test datasets and regression scripts. Continuously monitor quality and run A/B tests on prompts, tools, and model choices.

06

Combine Agents with Deterministic Code

Use traditional code for rules-based tasks; use agents for reasoning and ambiguous decisions. Hybrid designs maximize reliability and explainability.

Agents vs. RAG: When to Choose

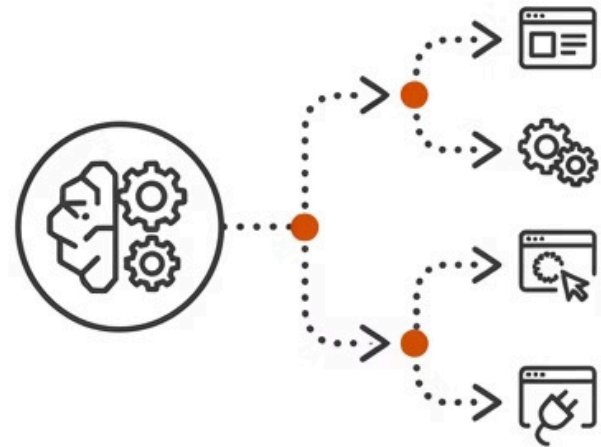
Both Agents and Knowledge Bases are powerful — but they serve different primary needs. The right choice depends on whether your workload is fundamentally about **information retrieval** or **task execution**.

RAG ONLY



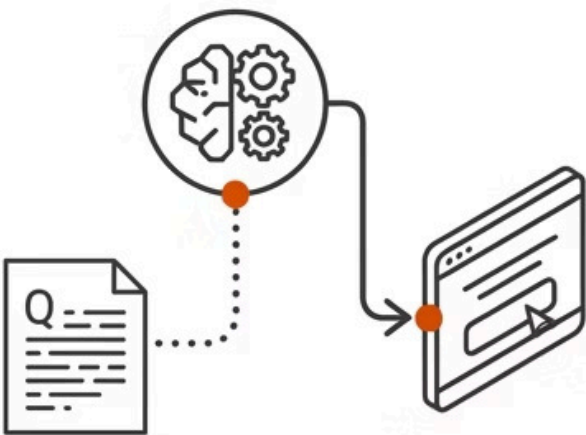
**Grounded Q&A.
Deterministic
Retrieval.**

AGENTS ONLY



**Task Orchestration.
Multi-step workflows.
API Integration.**

HYBRID AGENTS + KB



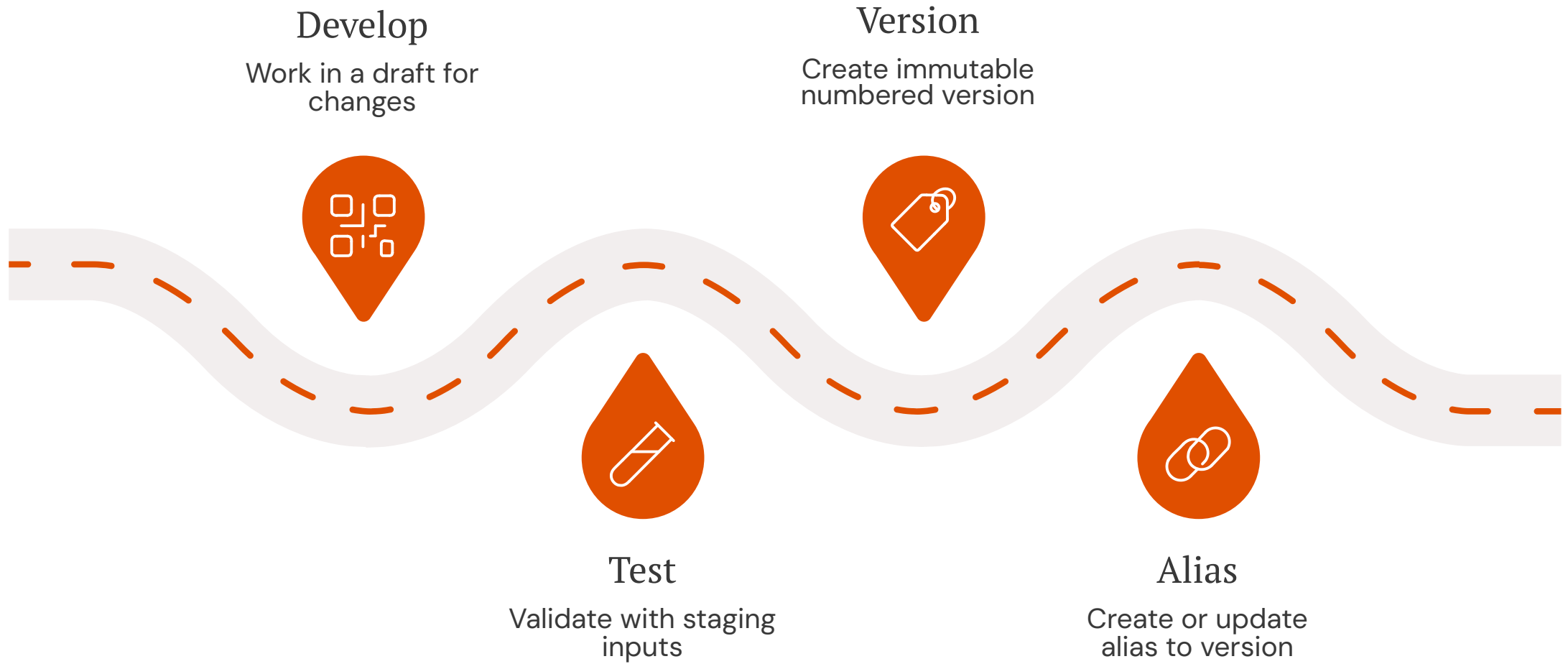
**Grounded Actions.
Customer Service /
Orders.**



Hybrid pattern: Attach a Knowledge Base directly to an agent to combine RAG grounding with tool-based action — for example, reading return policy documents while simultaneously modifying an order record.

Agent Deployment & Versioning

Bedrock Agents use a **draft** → **version** → **alias** promotion model that enables safe, zero-downtime deployments, easy rollbacks, and A/B testing across production traffic.



Versions

Numbered versions are immutable snapshots of the agent configuration. The working draft is always editable; versions are only created when you are ready to promote.

Aliases

Stable named pointers to specific versions. Switch versions by updating the alias — no application code changes required. Aliases support two states: `ACCEPT_INVOCATIONS` (active) and `REJECT_INVOCATIONS` (paused). Use multiple aliases for A/B testing or canary deployments.

Monitoring & Debugging

Robust observability is non-negotiable for production agents. Bedrock exposes native CloudWatch metrics alongside AgentCore's rich execution traces.

CloudWatch Metrics

InvocationCount / TotalTime

Total invocations and end-to-end wall-clock duration per agent call.

Token Usage & Throttles

InputTokenCount, OutputTokenCount, and throttled invocation counts for cost and capacity management.

TTFT / ModelLatency

Time to first token and time spent inside underlying model calls — key latency indicators.

Errors (4xx / 5xx)

Client and server errors broken out by type — essential for SLA alerting.

Debugging Strategies

- Enable verbose tracing on development aliases; disable for production to reduce overhead.
- Use separate test aliases and environments for staging — never test against prod aliases.
- Instrument token and tool usage per invocation for granular cost visibility.
- Set CloudWatch alarms on error rates, latency spikes, and throttling thresholds.
- Use AgentCore execution traces to refine prompts, instructions, and tool schemas iteratively.

Cost Optimization for Agents

Agent costs accumulate across model invocations, tool calls, Knowledge Base queries, and token consumption. A structured optimization strategy is essential for production workloads.

Model Invocations

Each reasoning and planning step invokes a Bedrock FM. Right-size model choice — use lighter models for routine tasks, frontier models for complex reasoning only.

Token Usage

Keep system and tool prompts concise. Avoid verbosity that inflates input tokens. Use Bedrock prompt caching for repeated prefixes on supported models.

Tool & KB Calls

Batch operations where possible. Cache frequent tool results within the session. Avoid redundant lookups by passing context between steps.

Service Tier Strategy

Priority / Reserved Tier

Use for latency-sensitive, high-SLA production traffic where consistent throughput is critical.

Flex Tier

Use for non-urgent background agent workloads, batch processing, and dev/test environments to significantly reduce cost.

- ❏ Always monitor token usage per agent invocation alongside tool call counts — these two levers account for the majority of agent cost at scale.

Key Takeaways for Workshop Content

1

Agent Capabilities

Emphasize multi-step reasoning, tool orchestration, and Knowledge Base integration as the core value proposition over simple chatbots.

2

AgentCore Suite

Present Runtime, Gateway, Identity, Memory, Code Interpreter, Browser, and Observability as the production-grade building blocks — not optional extras.

3

Design Patterns

Teach action groups, tool design, and prompt engineering best practices from the very first session. Bad tool design is the most common failure mode.

4

Multi-Agent Architectures

Show when and how to decompose into specialized agents. Demonstrate sequential, parallel, hierarchical, and collaborative patterns with concrete examples.

5

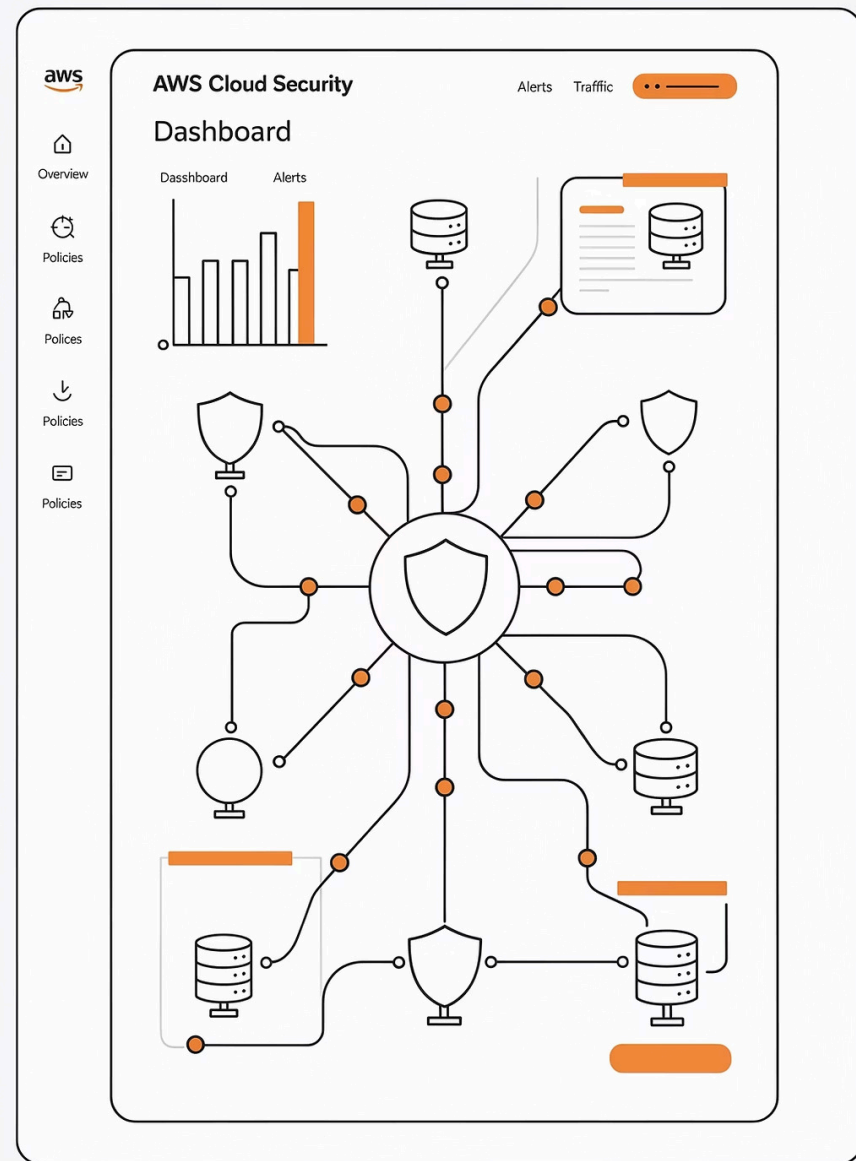
Monitoring & Cost

Stress observability, automated evaluation, and cost tracking as first-class engineering concerns — not afterthoughts. Build these in from day one.

Module 5: LLM Security and Observability on Bedrock

RESEARCH FINDINGS · FEBRUARY 2026

A deep-dive into Amazon Bedrock Guardrails, CloudWatch observability, security best practices, and cost optimization strategies for production LLM deployments. This module covers the current state of configurable safeguards, monitoring patterns, and practical guidance for security engineers building safer generative AI applications on AWS.





Guardrails: Content Filters

FEATURE 1 OF 6

Six Harm Categories

- **Hate** – Hateful or discriminatory content
- **Insults** – Demeaning or insulting language
- **Sexual** – Explicit or inappropriate content
- **Violence** – Graphic or violent content
- **Misconduct** – Illegal activities, self-harm
- **Prompt Attack** – Jailbreaks, injections, leakage

Filter Strength Levels

1 **NONE**
No filtering applied.

2 **LOW**
Blocks only high-confidence harmful content.

3 **MEDIUM**
Blocks high- and medium-confidence detections.

4 **HIGH**
Most aggressive — blocks all confidence levels.

Content Filter Tiers & Multimodal Support

Classic vs. Standard Tier (2025/2026)

Classic Tier

- Baseline content filtering
- Standard language support
- Lower cost option

Standard Tier

- Enhanced contextual understanding
- Typo and slang robustness
- ~60+ language support
- Improved prompt attack detection
- Distinguishes jailbreaks from injections

Multimodal Image Filters

Guardrails can evaluate both text and images in a single request, applying the same six harm categories to visual content. Public benchmarks indicate up to ~88% of harmful multimodal content can be blocked — treat as an indicative figure, not a hard guarantee.

Limits

- Max 20 images/request
- PNG and JPEG only
- Max 4 MB per image
- ~25 images/sec default rate limit

Guardrails: Denied Topics

FEATURE 2 OF 6

Denied Topics allow you to define custom subject-matter boundaries using natural language descriptions. Unlike content filters that target harmful language, denied topics enforce business-level constraints — preventing the model from engaging with entire domains regardless of how a prompt is phrased.

How It Works

- **Define topics in natural language**
No regex or keyword lists — describe what to avoid semantically.
- **Applied to inputs and outputs**
Policies fire on both user prompts and model-generated responses.
- **Standard tier for better boundaries**
Improved contextual understanding of topic edges reduces false positives.

Common Use Cases

Financial Advisory

Block advice beyond allowed scope or jurisdiction.

Medical Diagnosis

Restrict beyond high-level guidance; avoid clinical specifics.

Legal Counsel

Prevent case-specific or jurisdiction-specific legal advice.

Political Content

Block persuasion, campaign messaging, or partisan material.

Guardrails: Word Filters

FEATURE 3 OF 6

Capabilities

- Exact and pattern-based term matching
- Custom word and phrase lists
- Built-in profanity filtering
- Brand and IP protection controls

Word Filters provide surgical, deterministic blocking — useful when semantic filters are too broad or when you need to enforce strict brand or legal language policies.

Key Use Cases



Competitor Mentions

Prevent the model from referencing competing products or services by name in customer-facing outputs.



Confidential Codenames

Block internal project names, embargoed product codenames, and proprietary terms from appearing in any response.



Brand Language

Enforce approved terminology and block deprecated or off-brand phrasing across all model outputs.

Guardrails: Sensitive Information & PII Filters

FEATURE 4 OF 6

Guardrails can detect and act on personally identifiable information (PII) using ML-based entity recognition. Detection is probabilistic — monitor for false positives and negatives in production, especially across diverse input formats.

Supported PII Entity Types

Identity

Names, government IDs, SSN-style numbers, driver's licenses, passports, tax IDs.

Contact

Email addresses, phone numbers, physical addresses, IP addresses.

Financial

Payment card numbers and bank account numbers.

Policy Actions

BLOCK

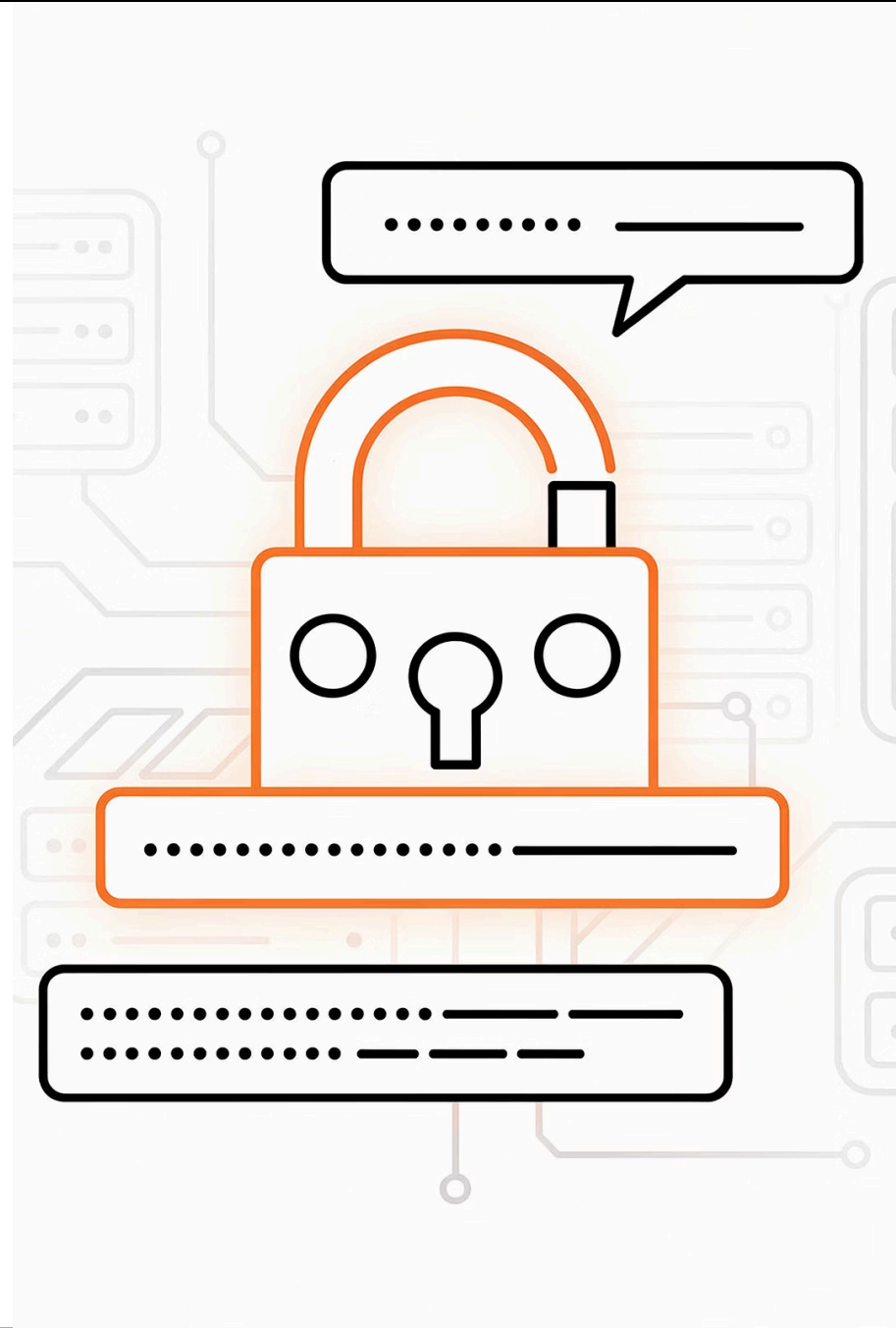
Reject the entire request when PII is detected. Use for zero-tolerance compliance environments.

ANONYMIZE

Replace PII with typed placeholders (e.g., <NAME>, <PHONE>). Allows processing to continue safely.

DETECT

Log and surface PII detections without modifying content. Ideal for evaluation and threshold tuning.



Guardrails: Contextual Grounding Checks

FEATURE 5 OF 6

Contextual Grounding helps detect hallucinations by verifying that model responses are supported by explicitly provided reference content. It is designed for RAG, summarization, and Q&A workflows — not open-ended conversational AI without a supplied reference document.

Grounding Check

Evaluates how strongly each sentence in the response is supported by the reference source or retrieved context. Assigns per-sentence grounding scores; responses below configured thresholds can be blocked or flagged before delivery.



Relevance Check

Evaluates whether the response is relevant to the original user query within the context of the provided source. Prevents off-topic answers even when the content is factually correct — a subtle but important distinction for RAG quality.

 **Scope & Limitations:** Best suited for summarization, paraphrasing, and document-grounded Q&A (e.g., RAG). Not intended for open-ended chat without a reference. AWS does not publish a specific hallucination reduction percentage — avoid citing "75% reduction" as a guaranteed figure.

Guardrails: Advanced Evaluation Capabilities

FEATURE 6 OF 6

What AWS Documents Today

The documented GA safeguards are **contextual grounding checks** and **content filters**. AWS is actively evolving evaluation capabilities, including:

→ LLM-as-judge for RAG evaluation

Using a model to score retrieval quality and answer faithfulness — currently in preview for Knowledge Bases.

→ Prompt attack detection improvements

Standard tier guardrails include enhanced jailbreak vs. injection disambiguation for better precision.

→ Ongoing reasoning-based research

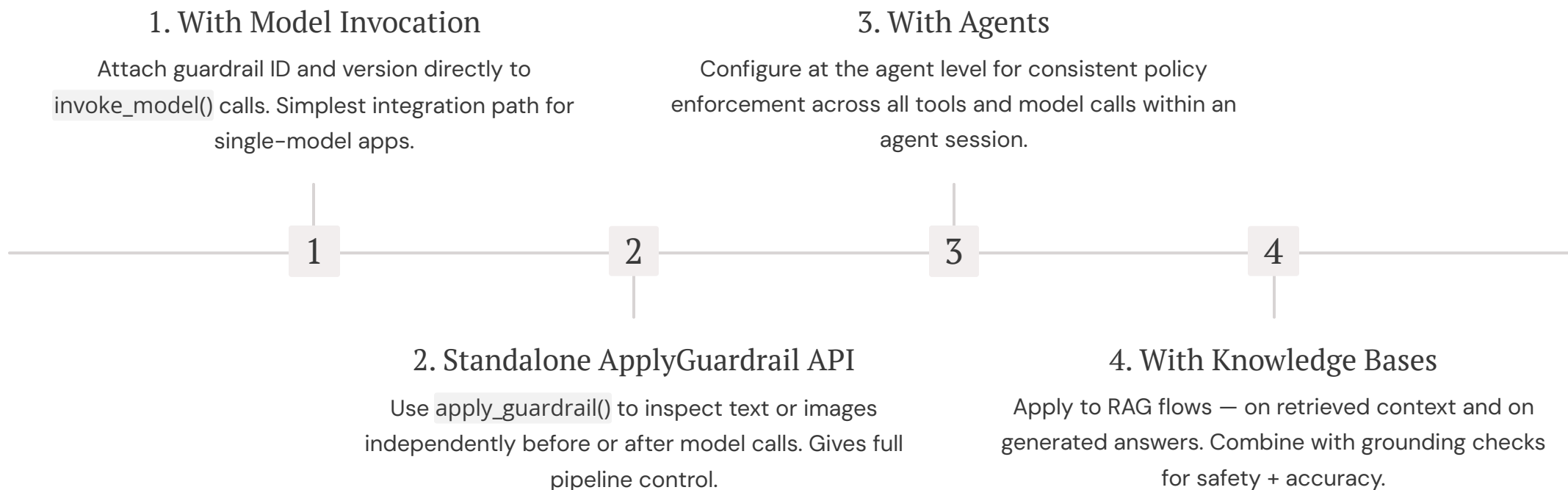
AWS continues to advance formal verification approaches — check current Guardrails and evaluation docs for GA features.

Accuracy Note

There is **no GA feature documented** as "Automated Reasoning Checks with sound mathematical techniques and 99% accuracy." Do not present hard claims like "first and only" or cite specific accuracy percentages for reasoning-based checks. Use conservative language and defer to current AWS documentation for latest feature status.

Guardrails: Configuration & Usage Patterns

Guardrails support three policy actions — **BLOCK**, **ANONYMIZE**, and **DETECT** — and can be applied across four integration patterns depending on where in your pipeline you need enforcement.



 **Tip:** The `source` parameter in `apply_guardrail()` accepts `'INPUT'` or `'OUTPUT'`, letting you run different policies at each pipeline stage independently.

Guardrails Best Practices

1 Start with DETECT Mode

Configure content filters, denied topics, and PII in DETECT mode first. Analyze logs to surface false positives and negatives before enabling BLOCK in production — avoid surprising users with unexpected refusals.

3 Prefer Standard Tier for Production

Better multilingual accuracy, stronger jailbreak robustness, and improved prompt attack precision. Worth the additional cost for external-facing applications where safety failures carry reputational or compliance risk.

2 Layer Multiple Safeguards (Defense in Depth)

Combine content filters + denied topics + PII detection + contextual grounding. No single guardrail is comprehensive; overlapping layers reduce the attack surface and catch what individual controls miss.

4 Adversarial Testing & Continuous Iteration

Build test suites targeting each risk category — toxicity, prompt leakage, jailbreaks, PII. Re-run after every major prompt, model, or policy change. Treat safety as an ongoing engineering discipline, not a one-time checkbox.

Observability: Model Invocation Metrics

CLOUDWATCH · AWS/BEDROCK NAMESPACE

Bedrock automatically emits CloudWatch metrics for every model invocation — no instrumentation required. These metrics are available per model, per operation, and per Region.



Performance

- **InvocationLatency** – Model response time
- **TTFT** – Time to first token (streaming)
- **TotalTime** – End-to-end processing
- **Invocations** – Request count



Token Usage

- **InputTokenCount** – Request payload tokens
- **OutputTokenCount** – Response tokens
- Aggregate per model for cost tracking



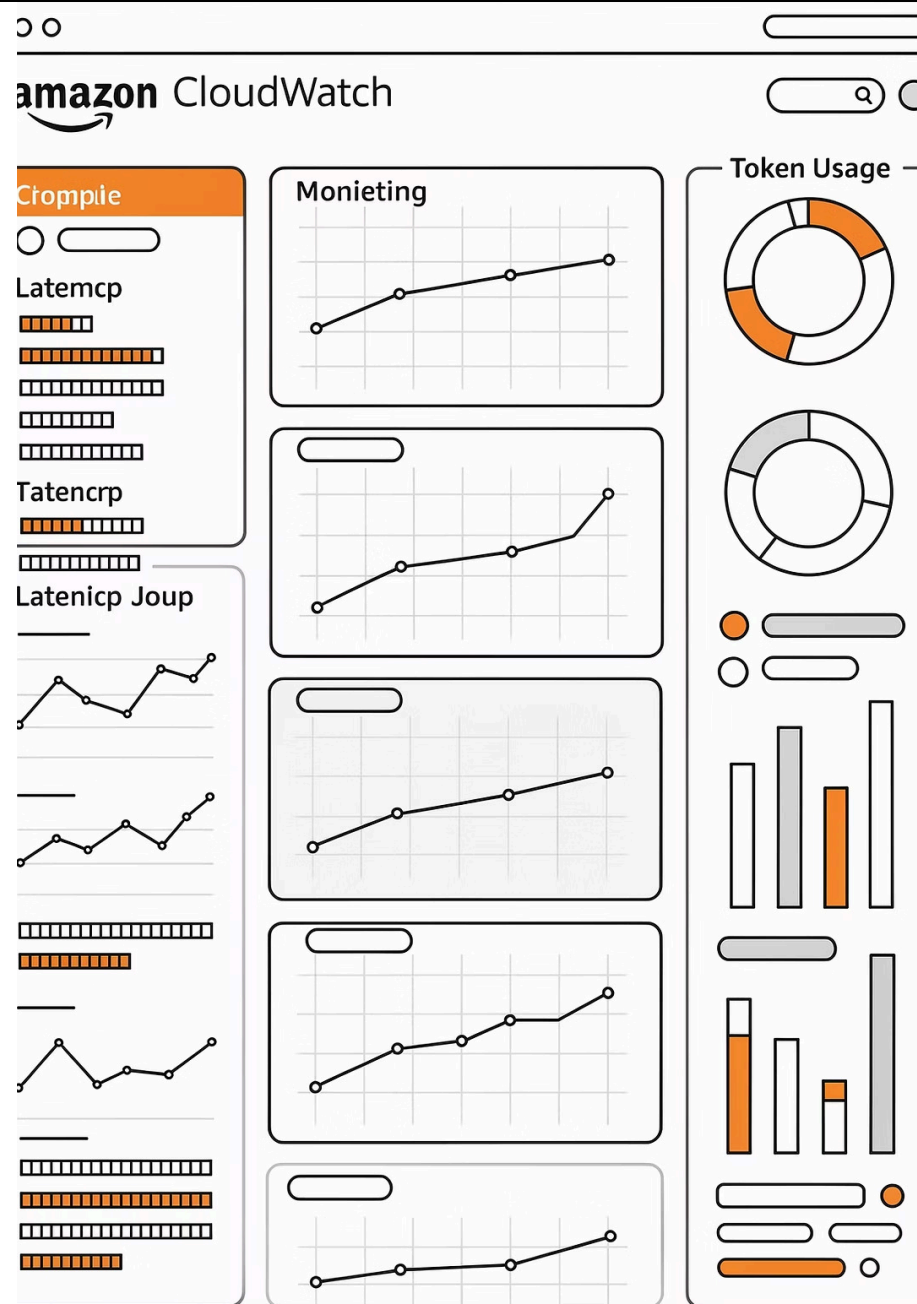
Errors & Throttling

- **InvocationThrottles** – Quota-limited requests
- **InvocationClientErrors** – 4xx issues
- **InvocationServerErrors** – 5xx failures



Key Dimensions

- **ModelId** – Per-model segmentation
- **Operation** – InvokeModel, Converse, etc.
- **Region / Account** – Cross-account views



Model Invocation Logging

PUTMODELINVOCATIONLOGGINGCONFIGURATION

Log Destinations

CloudWatch Logs

Real-time log streaming for debugging, anomaly detection, and Logs Insights queries.

Amazon S3

Long-term retention for compliance audits, offline analysis, and cost-effective archival.

Both (Recommended)

Dual-destination gives you real-time visibility and durable, queryable storage for production workloads.

What Gets Logged

- Request payloads — prompts and parameters
- Response payloads and metadata (model ID, timestamps, latency)
- Input and output token counts
- Guardrail decisions — which policy fired and on what content
- Error details for failed invocations

Always align logging configuration with your privacy, data residency, and compliance requirements before enabling full payload capture.

Agent & Knowledge Base Observability

Agent Metrics (CloudWatch)

CloudWatch metrics for Bedrock Agents are available across invocation, latency, and token dimensions. Key metrics include:

Invocation & Timing

InvocationCount, **TotalTime**, **TTFT**, **ModelLatency** — track end-to-end agent performance and isolate model call overhead.

Model Usage

ModelInvocationCount, **InputTokenCount**, **OutputTokenCount** — monitor per-call model consumption to identify runaway loops or inefficient orchestration.

Reliability

InvocationThrottles, **InvocationClientErrors**, **InvocationServerErrors** — dimensions include **AgentAliasArn** and **Operation** for precise segmentation.

Knowledge Base Logging

Ingestion Logs

Document ingestion status, chunking results, embedding generation. Debug format issues and parsing failures early.

Query Logs

Retrieval operations including queries, applied filters, and top-k results. Use for relevance tuning and query pattern analysis.

Error Logs

Ingestion failures, connectivity issues, and retrieval errors. Essential for SLA monitoring on RAG pipelines.

Batch Inference Monitoring

What to Monitor

- **Records processed** – Progress tracking for large jobs
- **Job status** – Lifecycle state transitions
- **Token throughput** – Approximate cost signal during execution

Metric names vary by API and model family — refer to the batch inference docs for exact metric identifiers for your configuration rather than hard-coding assumptions.

Operational Patterns

→ Set CloudWatch Alarms on Job Status

Trigger downstream workflows (e.g., Lambda, Step Functions) when batch jobs complete or fail — avoid polling.

→ Combine with S3 Output

Route batch results to S3 and use S3 event notifications for event-driven post-processing pipelines.

→ Track Cost via Token Throughput

Batch inference is typically ~50% cheaper than on-demand for eligible models. Monitor token metrics to validate savings.

CloudWatch Dashboards

1

Model Invocations

- Invocation counts per model over time
- p50 / p90 / p99 latency
- Input / output token usage by model
- Error and throttle rates

2

Agent Performance

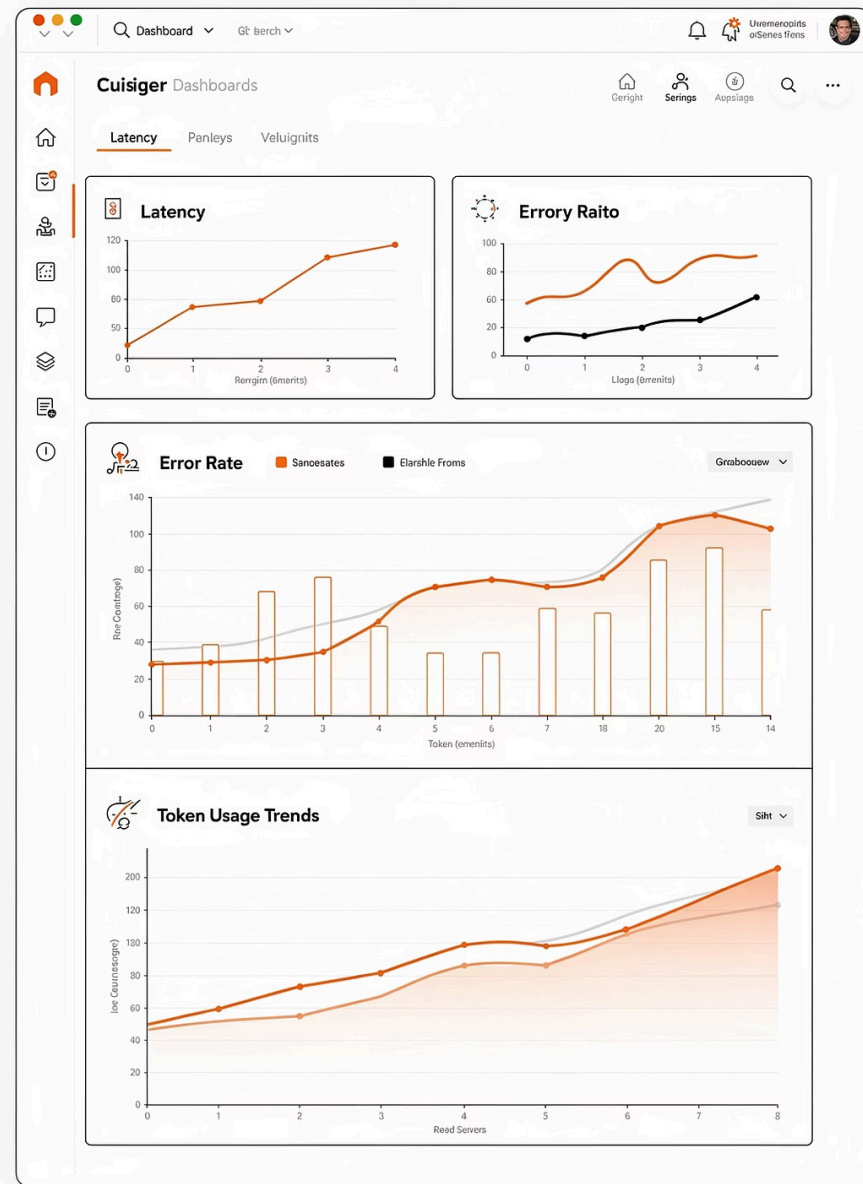
- Agent invocation trends by alias
- TotalTime and TTFT per alias
- ModelInvocationCount per agent call
- Token consumption and error patterns

3

Cost Monitoring

- Token usage per model and per agent
- Daily / monthly token trends
- Projected cost using Bedrock pricing
- Cost per request estimates

- 📌 **Dashboard Best Practices:** Combine performance, reliability, and cost metrics in one view. Segment by model, agent alias, and environment. Establish baselines early and track trends — not just point-in-time snapshots. Use log labels to separate tenants and teams.



CloudWatch Alarms

Define actionable alarms for each risk category. Alarms should route to SNS topics wired to on-call rotations or automated remediation workflows.



High Error Rate

Alarm when **InvocationClientErrors** or **InvocationServerErrors** exceed threshold over multiple evaluation periods to avoid noise from transient spikes.



High Latency

Alarm on p99 **InvocationLatency** or **TotalTime** above SLOs. Sustained high latency indicates capacity constraints or configuration issues.



Throttling

Alarm on sustained **InvocationThrottles**. Remediation options: reduce traffic, optimize batching, or request quota increases from AWS Support.



Cost / Token Anomalies

Use token metrics with CloudWatch or **Cost Anomaly Detection** to alert on unusual consumption — catches runaway loops, abusive use, or prompt engineering bugs.



Guardrail Violations

Derive custom metrics from logs for blocked / flagged events per policy type. Elevated rates signal attack campaigns or misconfigured thresholds requiring tuning.

CloudWatch Logs Insights Queries

ADJUST FIELD NAMES TO MATCH YOUR ACTUAL LOGGING SCHEMA

Find All Guardrail Blocks

```
fields @timestamp, modelId, inputText, guardrailAction
| filter guardrailAction = "BLOCK"
| sort @timestamp desc
```

Average Latency by Model

```
fields modelId, invocationLatency
| stats avg(invocationLatency) by modelId
```

Identify High Token Usage

```
fields @timestamp, modelId,
inputTokenCount, outputTokenCount
| filter outputTokenCount > 1000
| sort outputTokenCount desc
```

Track Error Patterns

```
fields @timestamp, errorCode, errorMessage
| filter ispresent(errorCode)
| stats count() by errorCode
```

📌 These query patterns assume standard log fields from `PutModelInvocationLoggingConfiguration`. If you embed metrics differently (e.g., via CloudWatch Embedded Metric Format), adjust field names accordingly. Always validate queries against a sample log set before wiring to production dashboards or alarms.

Security Best Practices

Data Protection

- TLS encryption in transit for all API calls
- KMS-managed or customer-managed keys at rest
- CMKs for stricter control and independent audit trails
- VPC endpoints (PrivateLink) to keep traffic inside AWS networks

Access Control

- Least-privilege IAM policies for Bedrock, S3, CloudWatch, Lambda
- Resource-based policies and SCPs for multi-account setups
- Session policies and short-lived credentials for applications
- AgentCore Identity for user-level agent access controls

Compliance

- ISO, SOC, CSA STAR, HIPAA eligibility, GDPR alignment
- FedRAMP High for eligible Regions and services
- CloudTrail for full API activity logging and audits
- Data residency via Region selection and in-region processing

Abuse Detection

- Guardrails prevent harmful content and proprietary data leakage
- Service-level rate limiting and quotas guard against runaway usage
- Logging and monitoring provide full audit trail for incident investigation

Cost Monitoring & Optimization

Cost Components

Model Inference

Per-token pricing for input and output tokens — typically the dominant cost driver.

Guardrails

Per-unit pricing for text safety, image filters, PII checks, and contextual grounding. Note: pricing was cut ~85% in late 2024.

Knowledge Bases

Vector storage and retrieval query costs; scale with index size and query volume.

Agents & Tooling

Model calls plus Lambda/tool invocation costs and any external API charges per agent turn.

Logging & Monitoring

CloudWatch Logs, metrics storage, and S3 archival — often underestimated at scale.

Optimization Strategies

1 Right-size models

Use lighter models for routine tasks; reserve frontier models for complex reasoning requiring high accuracy.

2 Optimize prompts

Reduce verbosity and minimize system message token overhead while preserving intended behavior.

3 Cache results

Use Bedrock prompt caching for repeated prefixes on supported models and cache tool results at the app layer.

4 Use appropriate service tiers

Flex for non-urgent workloads; Priority/Reserved for latency-sensitive or mission-critical traffic.

5 Batch processing

Up to ~50% cheaper than on-demand for eligible offline workloads.

Key Takeaways

MODULE 5 SUMMARY

1 Guardrails Are Essential for Production

Layered protections covering content, PII, and hallucinations must be turned on and tuned *before* going live — not after the first incident. Start in DETECT mode, then promote to BLOCK with evidence.

2 Defense in Depth — Never Rely on a Single Safeguard

Combine content filters, denied topics, PII detection, and contextual grounding. Each layer catches what the others miss; overlapping coverage is a feature, not redundancy.

3 Observability from Day One

Enable logging, metrics, and dashboards at project start. Retroactively adding observability to a running system is costly and introduces gaps. Visibility supports debugging, safety review, and cost governance simultaneously.

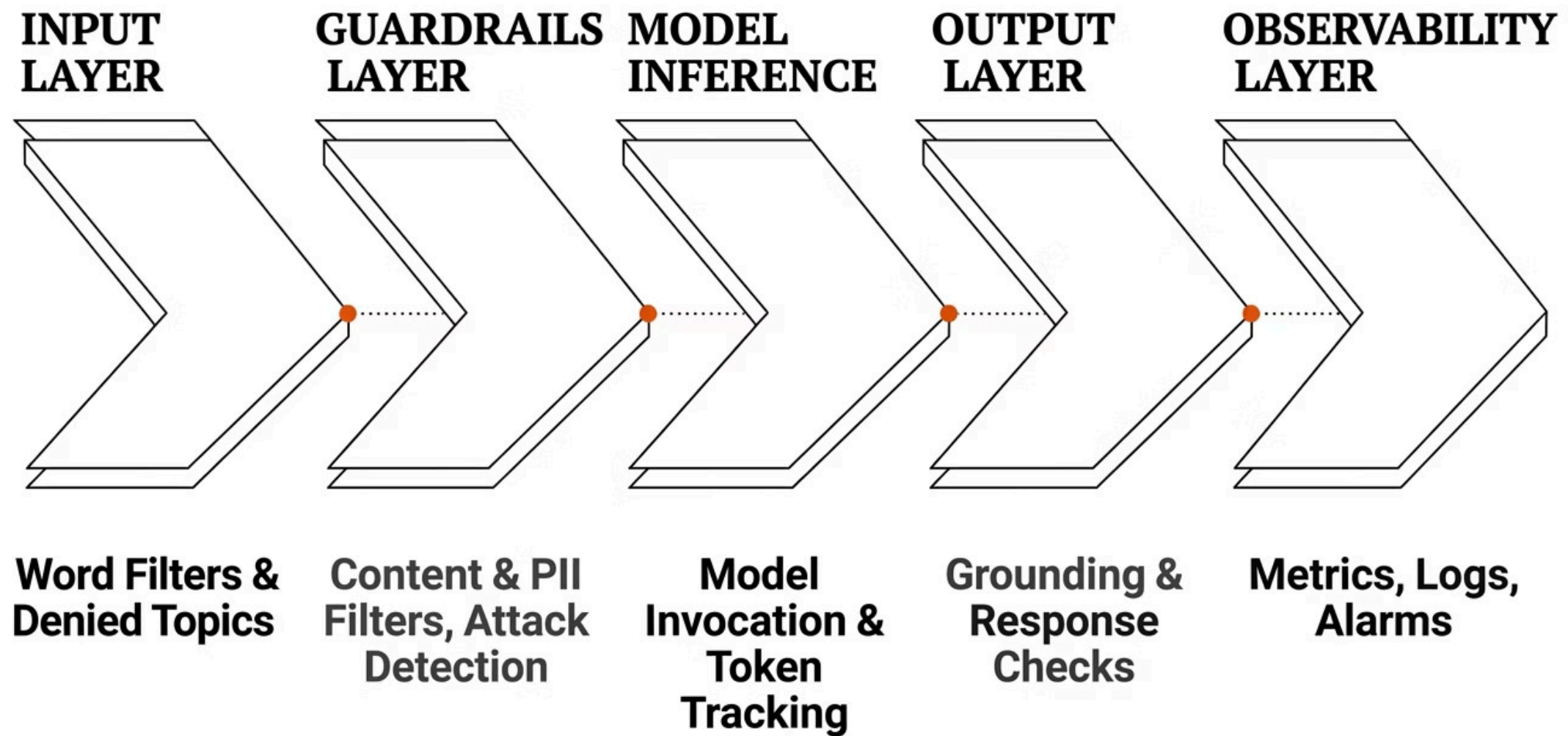
4 Continuous Tuning — Safety Is an Ongoing Process

Treat guardrail policies, denied topics, and PII configurations as living artifacts. Review after every major prompt, model, or traffic pattern change. Run adversarial test suites regularly.

5 Cost Awareness Is Non-Negotiable

Monitor token usage, guardrail invocations, and batch vs. on-demand patterns continuously. Use CloudWatch and Cost Anomaly Detection to catch runaway costs before they appear on the bill.

Reference Architecture Summary



This layered architecture reflects the defense-in-depth approach for production LLM applications on Bedrock. Each layer operates independently — enabling mix-and-match configuration — while CloudWatch ties the entire stack together with unified visibility across safety decisions, performance metrics, and cost signals.

Module 6: Cost Optimization

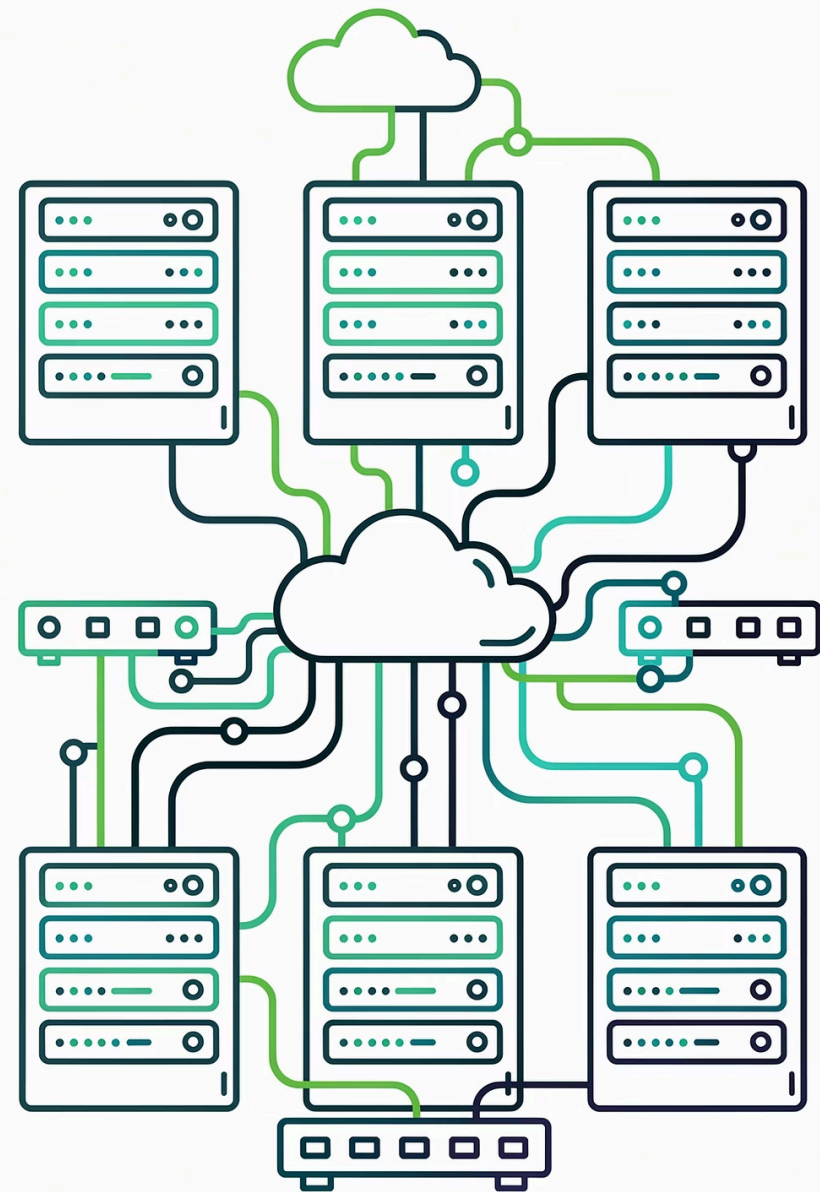
Amazon Bedrock — Research Findings · February 2026

Cost optimization for Amazon Bedrock spans model pricing, service tiers, batch vs. online inference, caching, and architectural choices across the full AI application lifecycle. This module provides a structured framework to help cloud architects, ML engineers, and technical product managers make informed decisions that balance performance, quality, and spend.

MODULE 6

AMAZON BEDROCK

COST STRATEGY



On-Demand Pricing: Service Tiers

Amazon Bedrock offers three on-demand service tiers that let you explicitly trade off price, availability, and performance to match each workload's requirements.

⚡ Flex Tier

~50% below Standard list prices. Best-effort, no strong SLA — highest throttling risk among tiers.

- Development, testing, and model evaluations
- Non-urgent batch and async workloads
- Agentic workflows that tolerate variable latency

⚖️ Standard Tier

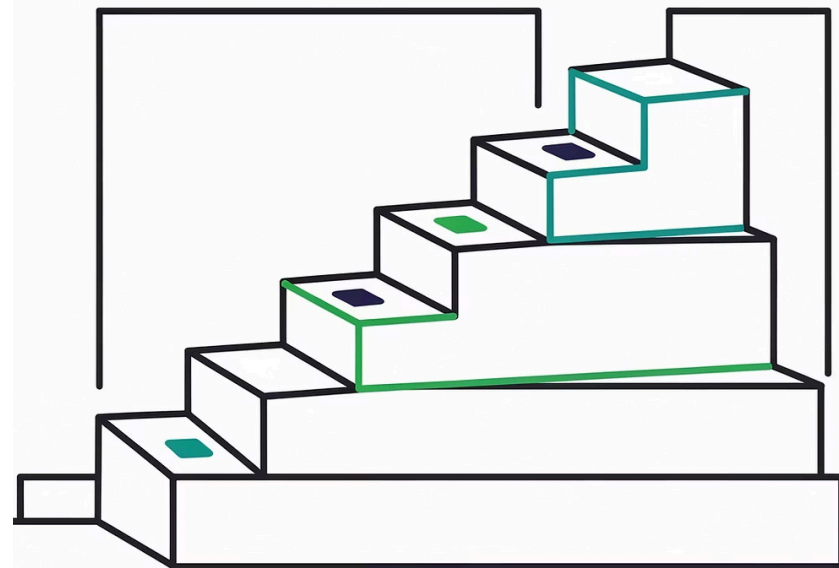
Baseline rates on the Bedrock pricing page. Balanced performance and throughput guarantees.

- General production applications
- Content generation, analysis, and RAG
- Consistent workloads with typical SLAs

🚀 Priority Tier

~75% above Standard. Preferential processing, higher capacity, and up to ~25% better latency/OTPS.

- Mission-critical, customer-facing apps
- Real-time translation and interactive UX
- Latency-sensitive paths where cost is secondary



Reserved / Provisioned Capacity

How It Works

Reserve capacity for specific models in advance for predictable pricing and guaranteed throughput. Commitment options include **1-month** and **3-month** (or longer) terms — longer commitments yield better effective rates.

Pricing is fixed for a configured throughput (e.g., tokens-per-minute or model units) for a given model, tier, and Region.

Break-Even Thinking

Reserved capacity often becomes cost-effective when you have **sustained, predictable usage**. A common internal heuristic: when average utilization is a substantial portion of peak — often cited as **>~40–50% vs. on-demand** — reserved options can start to pay off.

📌 This is not an AWS-documented threshold. Always run your own math per model and Region before committing.

Best For

- Consistent, high-volume production workloads
- Long-running apps with predictable traffic
- Teams where budget predictability is a hard requirement

Batch Processing: 50% Cost Savings

Key Characteristics

Cost Savings

~50% off Standard tier on-demand pricing for eligible foundation models in most Regions.

Processing Window

Jobs processed asynchronously; plan for a **24-hour completion window**, though many finish same-day.

Job Limits

Records per job, file sizes, and concurrent job limits vary by model and Region. Always consult the batch inference docs for current quotas.

Best Use Cases

- Training data generation at scale
- Content moderation backfills
- Bulk summarization and report generation
- Data enrichment and historical analysis where latency is not critical

📌 Batch inference is the single easiest cost lever for offline workloads — adopt it before exploring more complex optimizations.

Prompt Caching

Prompt caching reduces cost *and* latency when a static prompt prefix is reused across many calls — one of the highest-impact, lowest-effort optimizations available on Bedrock.

How It Works

Bedrock identifies and caches repeated **prompt prefixes** such as system prompts, long instructions, and shared context. On a cache hit, you pay only for **new input tokens (suffix) and output tokens** — cached prefix tokens are billed at heavily discounted or zero rates depending on the model's caching pricing. Cache management, storage, and expiry are handled automatically by Bedrock.

Supported Models

Initially available on select **Anthropic Claude** models; now extended to **select Amazon Nova** models and other families. Coverage is actively expanding — always verify per-model documentation.

Savings Potential

~90%

Savings on Cached Portion

For supported models when long system prompts are reused across many calls.

Best For

- Apps with stable system prompts or long shared instructions
- RAG systems with fixed instructions and repeated context structures
- Chatbots using a shared policy or prompt across many users

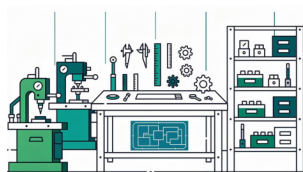
Model Selection and Right-Sizing

Choosing the right model class is the **highest-leverage cost decision** you make. Always start with the smallest model that meets your quality threshold, then scale up only where it delivers clear business value.



Light / Small Models

Compact Mistral, smaller Llama 3 family variants, and similar. **Fastest and cheapest** — ideal for classification, routing, basic Q&A, and high-traffic endpoints where quality demands are modest.



General / Mid-Range Models

Mid-sized Llama 3/3.1/3.2/3.3, mid-sized Nova models, Claude Sonnet. **Best balance** of quality, cost, and latency. Suitable for the majority of production workloads.



Flagship / Frontier Models

Claude Opus, larger Nova models, largest Llama variants. **Highest reasoning quality** at significantly higher per-token prices and latency. Reserve for tasks where accuracy directly drives business value.

📌 Indicative pricing example (subject to change, region-specific): Claude Sonnet $\approx$ \$3/1M input tokens, \$15/1M output tokens. Open-source models are often significantly cheaper per million tokens than proprietary frontier models. Always consult the Bedrock pricing page for current rates.

Model Distillation

Bedrock's model distillation workflow trains a **smaller "student" model** to mimic a high-quality "teacher" model — dramatically lowering serving cost and latency with minimal accuracy loss.

Measured Benefits (AWS Benchmarks)

5×

Faster Inference

500% improvement on many workloads
vs. the teacher model.

75%

Cost Reduction

Per-inference savings when serving the
distilled student model.

<2%

Accuracy Loss

On RAG-style tasks in internal tests —
often negligible in practice.

The Distillation Process

01

Select a Teacher Model

Choose a high-quality model such as Claude Opus or a large Nova variant.

02

Generate Training Data

Produce prompt → teacher-response pairs on your specific domain and use cases.

03

Run Bedrock Distillation

Use Bedrock's built-in distillation workflow to train the student model.

04

Deploy and Monitor

Serve the student in production; optionally fall back to the teacher for edge cases.

Best for: RAG apps, function-calling agents, and high-scale production workloads where serving cost and latency matter.

Intelligent Prompt Routing

Intelligent Prompt Routing automatically directs each prompt to the most appropriate model within a supported family — optimizing cost and quality *without changing application code*.

How It Works

A router model or policy classifies each incoming prompt by complexity and use case. **Simple prompts** are sent to smaller, cheaper models; **complex prompts** are escalated to larger, higher-quality models. Routing is typically transparent to application code — you call a family endpoint or routing API and the service handles the rest.

Key Benefits

- Cost savings without significant quality degradation
- Improved p95/p99 latency for simpler queries
- Reduced manual orchestration complexity in application code

Example Routing Pattern



Prompt Engineering for Cost Efficiency

Optimize for Clarity & Conciseness

Shorter, clearer prompts reduce token usage and often *improve* model behavior simultaneously. In practice, many real prompts can be shortened by **30–50% in tokens** while fully retaining intent.

Techniques:

- Remove filler words and redundancy
- Use direct language and explicit instructions
- Structure prompts with concise sections: role, context, task, constraints

❌ "I would like you to please help me understand what the weather is like today in New York City if you don't mind."

✅ "What's the weather in New York City today?"

Use Prompt Caching Effectively

Structure prompts so Bedrock's native caching delivers maximum savings on supported models.

Strategy:

- Put **static content first**: system prompt, policies, domain context
- Keep user-specific and dynamic content in the **suffix**
- Rely on Bedrock caching to avoid re-paying for the static prefix

❏ **Pattern:**

```
system_prompt = "" You are a helpful customer service agent. Company policies: [long policy document] ""
user_query = "What is your return policy on electronics?"
```

Few-Shot vs. Zero-Shot

Every example you add costs tokens. Be deliberate about when examples actually improve outcomes enough to justify the cost.

Zero-Shot

No examples. **Lowest token usage.** Always the starting point.

Few-Shot

Add examples only where they **significantly improve quality**. Keep examples minimal.

Consider distilling few-shot behavior into a student model to eliminate repeated example tokens at scale.

Service Tier Selection Framework

Use this decision framework to assign the right tier to each workload or traffic path — avoid over-provisioning with Priority when Standard or Flex will do.



Agent Design Optimization

Build Small, Focused Agents

One giant agent with 20+ tools, broad instructions, and long context leads to high token consumption and unpredictable behavior. Instead, decompose into **multiple specialized agents** with clear delegation.

✓ Best Practice

Specialized agents per domain (billing, orders, tech support) coordinated by a supervisor agent or routing logic. Short, domain-bounded prompts and efficient tool selection.

✗ Anti-Pattern

Single monolithic agent with 20+ tools, broad instructions, and unbounded context windows driving excessive token consumption.

Optimize Tool Descriptions

Clear, precise tool descriptions reduce unnecessary tool calls and costly retries. Every ambiguous description is a hidden tax on your token budget.

Include in every tool description:

- **What** the tool does and **when** to use it
- Expected **inputs** and **outputs**
- A simple inline example if the schema allows



✗ "Gets order info"



✓ "Retrieves order status, shipping info, and tracking number for a given order ID. Use when the customer asks about order status or shipment details."

Knowledge Base Optimization

Manage Data and Indexing Frequency

Embedding generation is typically a one-time cost per chunk — but re-embedding a large corpus unnecessarily can become a significant recurring expense. Vector storage costs also grow with chunk count and dimensionality.

- Use **incremental sync** to re-embed only changed documents
- Schedule large syncs during **off-peak hours**
- Avoid full-corpus re-embedding unless content changes broadly

Optimize Retrieval Settings

Every retrieval parameter is a cost lever. Higher top-k improves recall but increases token consumption linearly — tune based on evaluation, not intuition.

Top-k / Chunk Count

Start with **5–10 chunks** and tune based on answer-quality evaluation. Higher k = higher token + retrieval cost.

Reranking

Adds an extra model call but improves precision. Use when precision matters more than minimal cost; skip for highly cost-sensitive workloads.

Caching Strategies

Application-Level Caching

Cache frequently requested queries and responses at the application layer to eliminate redundant model invocations entirely. Particularly impactful for stable FAQ-style content and expensive RAG + generation flows.

What to cache:

- Frequent queries and their responses
- Static responses (e.g., common FAQs)
- Expensive RAG + generation flows with stable answers



Implementation sketch:

```
cached = cache.get(query_hash)
if cached: return cached
response = bedrock_runtime.invoke_model(...)
cache.set(query_hash, response, ttl=3600)
return response
```

Use Redis/ElastiCache for low-latency ephemeral caching;
DynamoDB or S3 for persistent caching.

Bedrock-Native Prompt Caching

Bedrock's built-in prompt caching works automatically for supported models once enabled — it's transparent to application logic. You simply structure prompts with reusable static prefixes and Bedrock handles the rest.



Structure First

Place all static content (system prompt, policies) at the start of the prompt prefix.



Combine Both Layers

App-level caching + Bedrock prompt caching together deliver maximum cost savings for high-traffic applications.

Batch Processing Strategy

The Core Value Proposition

Batch inference delivers approximately **50% cost savings** vs. Standard on-demand pricing for eligible models — with no infrastructure management overhead. Bedrock handles capacity utilization and job scheduling for you.

When to use it:

- Large-scale, non-interactive processing
- Historical data moderation and enrichment
- Bulk summarization and report generation
- Training data generation pipelines

Trade-Offs to Understand

✓ Batch

~50% cost savings. Scales to large job volumes. No worker fleet required. Predictable asynchronous processing within a 24-hour planning window.

⚡ Real-Time

Low latency and immediate responses. Required for interactive, user-facing workloads. Higher per-token cost at Standard or Priority tier rates.

📌 Batch inference is often the **fastest path to 50% savings** for teams with offline processing workloads. Evaluate before investing in more complex optimizations.

Monitoring and Cost Tracking

You cannot optimize what you do not measure. Instrument every Bedrock workload from day one — cost surprises at scale are painful and avoidable.

CloudWatch Metrics to Track

- **Token usage** per model, per tier, and per application
- **Request counts** and latency statistics (p50, p95, p99)
- **Error and throttle rates** — retries drive hidden token costs
- **Anomalous token consumption** — may indicate bugs or abuse

Set CloudWatch alarms for unexpected usage spikes, latency/error SLO breaches, and anomalous consumption patterns.

Cost Allocation Best Practices

Tagging Strategy

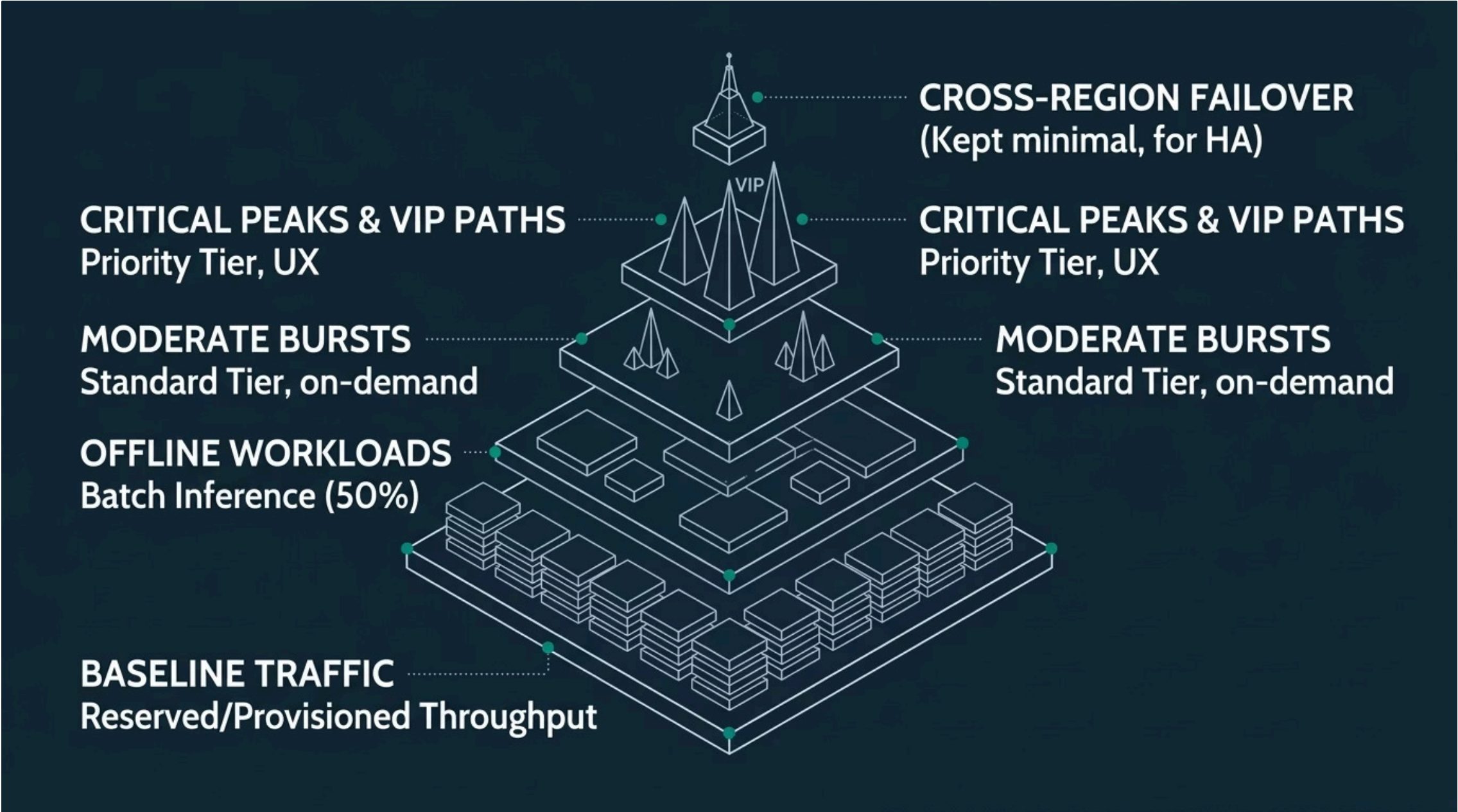
Tag every invocation by **environment** (dev/stage/prod), **team**, **project**, and **tenant** to enable per-team cost allocation in Cost Explorer.

Inference Profiles

Log an **application identifier** in each call to build per-app cost views. Combine CloudWatch, Cost Explorer, and logs for granular dashboards.

Combining Approaches: The Full Cost Strategy

No single lever delivers optimal results. A robust cost strategy layers multiple techniques across all traffic patterns simultaneously.



Layer reserved capacity at the base for cost predictability, add on-demand tiers for burst handling, and reserve Priority tier for the smallest slice of truly latency-sensitive traffic. Batch inference handles the long tail of offline workloads at half price.

Cost Monitoring Best Practices

1

Set Up Cost Dashboards

Track daily token usage by model and tier, cost per model and per application, and trend lines vs. budgets. Use Cost Explorer with tag-based filters for team-level visibility.

2

Implement Budget Alerts

Configure AWS Budgets and/or Cost Anomaly Detection with alerts at **50%, 80%, and 100%** of your monthly budget. Pair with CloudWatch/SNS for real-time notification to on-call engineers.

3

Conduct Regular Cost Reviews

Weekly or monthly reviews of top cost drivers. Systematically evaluate model choices, tier assignments, caching effectiveness, and batch conversion opportunities before committing to changes.

4

Run A/B Tests for Optimization

Test different models, prompt variants, caching levels, and tier assignments side-by-side. Measure **cost per request vs. quality vs. latency** to make data-driven decisions — not assumptions.

Real-World Cost Optimization Patterns

These illustrative examples show how layering multiple strategies produces compounding savings. Treat them as patterns to adapt, not official case studies.

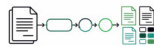


Customer Service Chatbot

Before: Frontier model (Claude Opus) for all queries, Priority tier, no caching — ~\$5,000/month.

After: Mid-tier model (Claude Sonnet/smaller Nova) for most queries; Intelligent Prompt Routing escalates complex cases to Opus; Standard tier for most traffic; prompt + app-level caching for FAQs.

Result: ~\$1,200/month (~75–80% savings)



Document Processing Pipeline

Before: Real-time per-document inference, Standard tier only, no batch — ~\$8,000/month.

After: Bulk summarization and moderation moved to batch inference (50% rate), jobs scheduled off-peak, smaller classification model for simple docs.

Result: ~\$2,800/month (~65% savings)



RAG Application

Before: Large model for all queries, 20 chunks retrieved each time, no caching — ~\$3,500/month.

After: Intelligent Prompt Routing + mid-tier model; prompt caching for shared instructions; top-k reduced from 20 to 8 after evaluation; frequent-query result caching.

Result: ~\$900/month (~70–75% savings)

Key Takeaways

Achieving meaningful Bedrock cost savings requires a multi-dimensional, iterative approach — not a single switch to flip.



Multi-Dimensional Optimization

Combine model selection, service tiers, batch inference, prompt caching, and architecture patterns. No single lever achieves optimal savings alone.



Measure Everything

Token usage, latency, error rates, and per-application costs are *required inputs*, not nice-to-haves. Instrument from day one.



Iterate Incrementally

Start simple, instrument, then optimize models, prompts, and tiers based on real data. Avoid premature optimization before you have usage baselines.



Quality vs. Cost

Use structured experiments and RAG/agent evaluations to find the right balance. The cheapest model that meets your quality threshold is the right model.



Automate Cost Efficiency

Leverage Intelligent Prompt Routing, model distillation workflows, and batch scheduling to make cost-efficiency systematic rather than manual.